



UNIVERSITÉ CATHOLIQUE DE
LOUVAIN
ECOLE POLYTECHNIQUE DE
LOUVAIN



Plateforme web pour accompagner l'apprentissage de la programmation par les 10 - 14 ans

Promoteurs :
Chantal PONCIN
Pierre SCHAUS
Lecteur :
Kim MENS

*Mémoire présenté en vue de
l'obtention du grade de
master ingénieur civil en informatique
option software engineering and
programming systems
par Simon CLAESSENS
et Gaëtan COLLART*

Louvain-la-Neuve
Année académique 2013 - 2014

Plateforme web pour accompagner l'apprentissage de la programmation par les 10 - 14 ans

Mémoire - LINGI2990

Étudiants :

Gaetan COLLART	Simon CLAESSENS
<i>Année</i> : INFO 22 MS	<i>Année</i> : INFO 22 MS
<i>Noma</i> : 66-20-07-00	<i>Noma</i> : 31-06-07-00

Promoteurs :

Chantal PONCIN	Pierre SCHAUS
----------------	---------------

2 juin 2014

Résumé

Ordinateurs, smartphones, tablettes, montres connectées, voitures intelligentes; de nos jours, les technologies informatiques sont partout. Les exemples dans la vie quotidienne sont innombrables. En revanche, les personnes qui comprennent leur fonctionnement deviennent de plus en plus rares.

La programmation prend de plus en plus de place dans notre vie. Or, quelle place est laissée à son apprentissage dans les programmes officiels? De plus, l'apprentissage de la programmation pourrait être bénéfique pour les jeunes. En effet, la pratique de la programmation forge une logique et un esprit analytique.

L'idée poursuivie dans ce mémoire est de fournir une plateforme d'apprentissage de la programmation orientée vers les écoles. Ce travail présente une plateforme d'apprentissage de la programmation articulée autour de missions ludiques et de défis créatifs.

La plateforme est basée sur des technologies web dans le but de la rendre la plus portable possible. Elle intègre un environnement de programmation par blocs pour permettre à tous de rentrer rapidement dans les activités. De plus, cette plateforme est entièrement en français ce qui n'existait pas encore actuellement.

Des enfants lors d'expériences ont eu l'occasion d'utiliser cette plateforme ce qui a permis de tester de manière pratique cette application. Elle a rencontré un franc succès, ce qui laisse à penser que cette solution fonctionne.

Abstract

Computers, smartphones, tablets, connected watches, smart cars ; nowadays, computer technologies are everywhere. Examples in daily life are countless. However, the people who understand how those technologies work become more and more rare.

Programming has become more and more important in our lives. Yet, which place is reserved for this learning in the official programs? In addition, learning programming could be beneficial for young people. Indeed, the practice of programming trains a logical and analytical mind.

The idea pursued in this paper is to provide a school-oriented programming platform. This work presents a platform relying on fun missions and creative challenges.

The platform is based on web technologies in order to make it as portable as possible. It includes a blocks-based programming environment that allows everyone to begin the activities quickly. In addition, this platform is entirely in French which has never been done up to now.

Children had an opportunity to test this platform during experiments which allowed to test this application in a practical way. It was a complete success which suggests that this solution works efficiently.

Remerciements

Dans cette partie les auteurs tiennent à remercier tout particulièrement certaines personnes et organisations.

Le projet Snap! BYOB (Snap!) et toutes les personnes qui lui donnent vie. Sans ce projet ou sans son caractère open source ce mémoire n'aurait pas été aussi abouti.

Poncin Chantal et Schaus Pierre pour nous avoir guidé et soutenu tout au long de ce mémoire.

Tous les lecteurs : Claessens Yves, Collart Eric, Deltour Christine, Poncellet Thibault, Questiaux Isabelle ; pour avoir relu et permis de réduire le nombre de fautes d'orthographe, d'améliorer le style rédactionnel et la qualité technique.

L'initiative KidsCode pour nous avoir accueilli, permis de mener une expérience chez eux et pour leur compte rendu de cette expérience qui nous a bien orientés.

L'initiative Science Infuse qui nous a permis de mener des expériences avec 4 classes d'enfants ce qui fut très bénéfique à la qualité de la solution qui est proposée.

Table des matières

1	Introduction	1
1.1	Contexte	1
1.2	Problème	1
1.3	Motivation	2
1.4	Objectifs	2
1.5	Approche	3
1.6	Contributions	3
1.7	Plan du mémoire	4
2	Connaissances nécessaires	5
2.1	Prérequis	5
2.2	Snap!	6
2.2.1	Utilisation	6
2.2.2	Blocs	7
2.2.3	Programme	9
2.2.4	Interface graphique	9
2.3	Ruby on Rails	9
2.3.1	Philosophie	12
2.3.2	Architecture	12
2.3.3	Gems	14
2.3.4	Tests	14
3	Travaux associés	17
3.1	État de la programmation dans le monde	17
3.1.1	Angleterre	17
3.1.2	France	18
3.1.3	Corée du Sud	18
3.1.4	Grèce	18
3.1.5	Nouvelle-Zélande	18
3.1.6	Belgique	18
3.2	Organisations existantes	19
3.2.1	Code.org	19
3.2.2	CoderDojo	20
3.2.3	Code Club	21
3.2.4	KidsCode	21
3.3	Langages de programmation	22
3.3.1	Blockly	22
3.3.2	Scratch	22
3.3.3	Snap!	24

3.3.4	Python	24
3.4	Concepts différenciateurs	25
3.4.1	Pour qui ?	25
3.4.2	Comment ?	25
3.4.3	De qui ?	27
4	Définition de la problématique	29
4.1	Positionnement de Rsnap	29
4.1.1	Âge, genre et origine des utilisateurs	29
4.1.2	Outils, concepts et environnement de travail	29
4.1.3	Type d'organisation, enseignants, création des cours	30
4.2	Choix technologiques	31
4.2.1	Critères d'utilisations	31
4.2.2	Plateforme d'apprentissage	32
4.2.3	Langage d'apprentissage	32
5	Présentation de la plateforme Rsnap	35
5.1	Missions	35
5.1.1	Schéma général	35
5.1.2	En voiture	36
5.1.3	L'hélicoptère	37
5.1.4	Soyons courtois	38
5.1.5	Tu ne m'attraperas jamais	38
5.2	Snap!	39
5.2.1	L'interface	39
5.2.2	Les rôles	40
5.2.3	Masquer les scripts	40
5.2.4	Traduction	41
5.2.5	Sauvegarde sur le serveur	41
5.3	Rsnap	41
5.3.1	Implémentation	41
5.3.2	Utilisation	46
6	Validation	53
6.1	Implémentation	53
6.1.1	Tests unitaires	53
6.1.2	Utilisation de métriques	56
6.2	Expériences	57
6.2.1	KidsCode	57
6.2.2	Printemps des Sciences	59
6.3	Formulaires	62
6.3.1	Création du formulaire	62
6.3.2	Protocole de récolte	64
6.3.3	Analyse des formulaires	64

6.3.4	Avis général sur les expériences	68
7	Conclusions	69
7.1	Plateforme d'apprentissage	69
7.2	Contenu	70
7.2.1	Missions existantes	70
7.2.2	Création de missions	70
7.3	Langue	71
7.4	Bilan final	71
8	Futures améliorations	73
8.1	Tester la réussite des missions	73
8.2	Aide à la gestion des groupes	73
8.3	Améliorations et ajout de missions	74
8.4	Blocs d'introspection des programmes	74
8.5	Charte graphique	75
8.6	Futur de l'application	75
8.6.1	Amélioration du code	75
8.6.2	Amélioration de l'utilisabilité	76
	Glossaire	77
	Bibliographie	79
A	Référence	83
A.1	Utilisation de Rsnap	83
A.1.1	Créer une mission	83
A.1.2	Ordonner les missions	85
A.1.3	Corriger une mission	86
A.1.4	Réaliser une mission	87
A.1.5	Réaliser un projet libre	89
A.2	Réponses au formulaire des enfants	89
A.3	Réponses au formulaire des professeurs	89

Introduction

Ce travail présente une plateforme d'apprentissage de la programmation appelée Rsnap. Cette plateforme est destinée à soutenir l'apprentissage de la programmation chez les enfants de 10 à 14 ans.

Ce chapitre introductif commence avec une mise en contexte du problème suivi d'une brève description de celui-ci, vient ensuite les motivations qui ont incitées à la réalisation de ce mémoire. Les objectifs sont abordés ensuite, suivis de l'approche choisie. Ce chapitre se termine avec les contributions que ce travail apporte à l'état de l'art et le plan du mémoire.

1.1 Contexte

Dans un monde où la pédagogie est un sujet de société et où l'informatique est omniprésente, on s'étonne de n'observer que peu d'interactions entre ces deux sujets. Il existe toutefois quelques recherches et initiatives. Il faut encore distinguer celles qui concernent l'informatique dans son ensemble et celles qui se portent sur la programmation.

Il existe deux écoles d'apprentissage de la programmation. La première prône l'apprentissage de la programmation directement avec des langages de programmation classique. D'autres préfèrent des langages simplifiés ; par exemple la programmation par blocs.

Ces deux écoles se différencient essentiellement par rapport au public qu'elles visent. Une programmation par blocs sera plus simple dans un premier temps et donc sera mieux adaptée aux enfants plus jeunes n'ayant aucune expérience avec la programmation. Une approche avec un langage complet comme Python sera plus adaptée pour des enfants plus âgés ou ayant déjà une expérience de la programmation.

Des initiatives sont disponibles dans d'autres pays que la Belgique et d'autres langues. Mais aucune de ces initiatives n'a été conçue pour un public francophone.

1.2 Problème

Comme introduit dans la section précédente, il y a un manque de soutien à l'apprentissage de la programmation pour les jeunes francophones. Il n'y

a pas ou peu de moyen pédagogique actuellement. Ce manque de moyen n'incite pas ceux qui souhaiteraient prendre le train en marche et commencer à enseigner la programmation

Il y a donc un besoin pour une plateforme spécialisée dans l'apprentissage de la programmation accessible à tous. Il n'est pas réaliste de demander au professeur d'être programmeur. Il est donc nécessaire que cette plateforme soit utilisable sans avoir de connaissances particulières dans ce domaine.

1.3 Motivation

La programmation est partout, pourtant bon nombre de personnes ne comprennent pas comment cela fonctionne. Il est intéressant que les enfants aient une idée du fonctionnement des objets de leur quotidien.

L'éducation des jeunes à la programmation peut être bénéfique à la société sur de nombreux points, comme : une structuration logique de l'esprit, une meilleure compréhension de l'environnement ou encore un meilleur départ pour les enfants qui souhaitent se lancer dans l'informatique.

L'Europe [1] nous indique que la programmation est un très bon exercice pour l'acquisition d'un raisonnement logique. En effet, la programmation demande beaucoup de rigueur. Une erreur, un oubli dans la structuration ou dans la syntaxe entraîne directement un bogue.

Dans les cours de technologie, par exemple, plusieurs professeurs essayent d'apprendre l'informatique aux jeunes. Mais par manque de support, d'applications adéquates et probablement aussi par manque de connaissance dans ce domaine, ils n'enseignent que trop peu la programmation et se contentent de la bureautique. Ce travail veut fournir une aide pour tous ces professeurs en mettant à leur disposition une plateforme qui les soutiendrait dans leur enseignement.

1.4 Objectifs

Ce travail poursuit trois objectifs principaux :

Plateforme d'apprentissage : Fournir une plateforme qui permet d'enseigner la programmation aux jeunes de 10 à 14 ans. Cette plateforme doit être disponible partout et être facile d'utilisation. Elle doit également pouvoir stocker des informations pour qu'elles soient disponibles lors de prochaines utilisations.

Création de contenu : Fournir du contenu à la plateforme définie plus haut. A savoir des exercices qui ont pour but d'introduire la programmation chez les jeunes. Un contenu initial est très important pour servir d'exemple à la création de futurs contenus.

Langue : Un troisième objectif déjà annoncé : ce travail est à destination d'un public francophone, il est donc primordial que la plateforme ainsi que le contenu soient en français.

1.5 Approche

L'application développée dans le cadre de ce travail se nomme Rsnap [2].

La plateforme : La plateforme sera implémentée par une combinaison entre un environnement de programmation existant Snap! [41] et un site web en Ruby on Rails (Rails) [28]. Cette combinaison permet d'avoir une plateforme accessible par tous les ordinateurs ayant accès à internet et un navigateur récent.

La plateforme devant aussi permettre la création de contenu par les professeurs, il sera nécessaire d'introduire des rôles différents pour ceux-ci et les jeunes.

Le contenu : Le contenu initial fourni sera une série de missions qui se succèdent, la fin de l'une débloquent la suivante. Cette série d'exercices introduira les concepts nécessaires à la création d'un jeu plus conséquent. Ce jeu sera un jeu de poursuites permettant aux jeunes de jouer entre eux.

1.6 Contributions

Cette section présente les différentes contributions de ce travail. Les trois principales sont : un outil francophone pour apprendre la programmation, une plateforme intégrée orientée pour l'enseignement et la combinaison d'outils existants.

Un outil francophone : Comme introduit dans la section 1.2 il n'existait pas encore ou peu de matériel francophone pour soutenir l'apprentissage de la programmation. Rsnap est une plateforme entièrement francophone.

Une plateforme d'apprentissage : Aucune plateforme n'est actuellement disponible pour l'apprentissage de la programmation. Rsnap se veut être orienté pour l'apprentissage dans l'enseignement traditionnel. Ceci en intégrant une gestion de classe, une progression contrôlable par l'enseignant. Rsnap a été pensée dès le début pour que les élèves ainsi que les professeurs puissent l'utiliser de la manière la plus autonome possible.

Combinaison d'outils existants : Ce travail montre que de nombreuses applications sont déjà disponibles. Malheureusement, elles sont souvent restreintes à leur domaine d'application tel que les langages informatiques ou les plateformes web. La force de ce mémoire a été de combiner ces applications afin qu'elles se complètent pour devenir une application plus vaste.

1.7 Plan du mémoire

La suite de ce mémoire se divise de la manière suivante :

Connaissances nécessaires Ce chapitre introduit le bagage technique nécessaire à la bonne compréhension de ce travail. Il commence par quelques prérequis, puis introduit Snap! et enfin parle plus en détail de Rails.

Travaux associés Ce chapitre présente l'état de l'art dans le monde et l'avancée de l'apprentissage de la programmation dans le monde. Ensuite, il présente quelques grandes initiatives similaires à celle de ce travail. Après quoi, il introduit les différents langages de programmation enseignés aux jeunes et se termine par une liste de concepts différenciateurs qui permet de définir une initiative de manière unique.

Définition de la problématique Ce chapitre présente la position de ce mémoire par rapport au chapitre précédent. Il positionne Rsnap par rapport aux concepts différenciateurs introduits dans les travaux associés. Et pour finir, il décrit les choix technologiques faits dans le cadre de ce travail.

Solution Ce chapitre décrit la plateforme Rsnap implémentée dans le cadre de ce mémoire. Il commence par décrire les missions qui ont été créées pour ce travail. Il approche ensuite des modifications apportées à Snap! pour les besoins du mémoire. Il finit par décrire la partie web de Rsnap.

Validation Ce chapitre présente la validation de l'application. Il commence par les tests sur l'application Rsnap. Il présente ensuite les expériences qui ont été menées. Il se termine par l'analyse des enquêtes de satisfaction proposés aux participants de l'expérience.

Conclusion Ce chapitre compare les objectifs attendus et atteints, les points plus faibles et les points forts de ce mémoire.

Futures améliorations Ce chapitre décrit ce qui serait utile de faire dans la continuité de ce travail. Ce sont des améliorations qui n'ont pas pu être réalisées, soit par manque de temps, soit parce qu'elles sont apparues dans l'analyse des expériences.

Connaissances nécessaires

Ce chapitre commencera par présenter de manière succincte les prérequis pour pouvoir suivre ce document. Ensuite, une description plus technique présentera les outils utilisés.

2.1 Prérequis

Les connaissances nécessaires à la bonne compréhension de ce travail vont être développées dans cette partie.

Snap ! Commençons par la partie portant sur Snap! BYOB (Snap!). Cette application est implémentée en JavaScript. Donc, de bonnes connaissances dans ce langage de programmation seront un atout pour comprendre les logiciels utilisés et les apports et modifications apportées à ceux-ci.

Cette application utilise une programmation dite par blocs. Le principe est d'empiler des blocs qui représentent des opérations. Une fois ces blocs assemblés, ils représentent un programme exécutable. Un exemple de programme est disponible sur la figure 2.1

Ruby on Rails La plateforme web est implémentée en Ruby on Rails (Rails) qui est le framework le plus utilisé pour la création de site web en Ruby.

Des connaissances sur l'architecture modèle-vue-contrôleur (MVC) sont nécessaires.

L'utilisation de gems, bibliothèque logicielle, demande des bases en développement Ruby. Plus de détails techniques seront donnés dans la suite de ce chapitre 4.2.

Pédagogie Pour l'apprentissage de la programmation aux jeunes, des notions de pédagogie sont également nécessaires. Les programmes d'apprentissage existants permettent de définir pour chaque année d'étude, les concepts considérés comme acquis par les élèves. La communication avec les jeunes demande un vocabulaire adapté. Tout ceci s'avère aussi important pour les choix de design d'interface. Par exemple, suivant leur âge, les jeunes ne seront pas stimulés par les mêmes images ou couleurs [7].

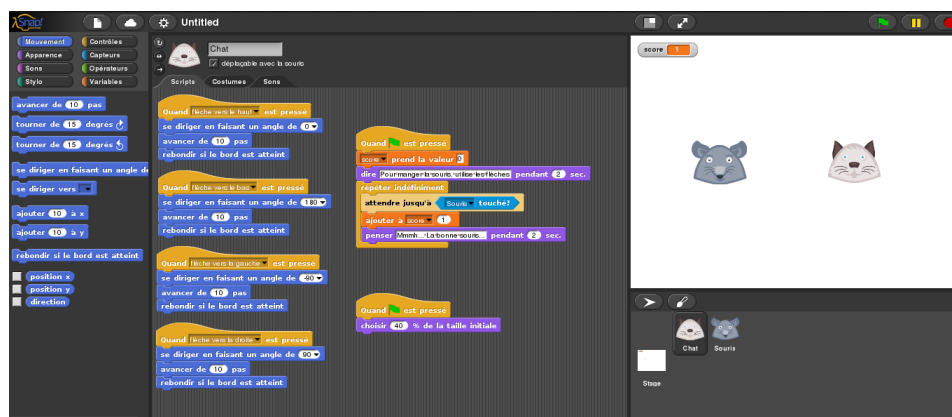


FIGURE 2.1 – Exemple de programmation par bloc

2.2 Snap !

Cette partie donne un aperçu de l'application qui est utilisée dans ce travail. La présentation de Snap! [41] se divise en plusieurs sections. La première explique les différents types de blocs et comment ils sont implémentés. L'étape suivante explique l'exécution d'un programme. Ensuite vient l'affichage des différents éléments de Snap!.

2.2.1 Utilisation

Cette partie va illustrer comment réaliser un programme avec Snap!. Ceci permet de découvrir les différentes zones de l'interface de Snap! et leurs utilisations. Cette explication se base sur les images 2.2 et 2.1.

Personnages Pour réaliser un programme avec Snap!, il faut commencer par savoir combien de personnages sont souhaités. Les personnages s'appellent des *sprites* et sont visibles dans la *zone des sprites* de la figure 2.2. On peut voir dans le programme de l'image 2.1, que deux personnages sont présents : un chien et un chat. Le bouton blanc au dessus de la *zone de sprites* permet d'ajouter des personnages.

Scripts Après avoir ajouté le nombre de personnages souhaité, il faut maintenant faire des scripts qui seront le code du programme. Les scripts sont assemblés, par glisser-déposer, dans la *zone d'édition* au centre de l'interface. Ceux-ci sont construits à l'aide de blocs qui se trouvent dans la *zone des blocs sources*. Cette zone contient tous les blocs disponibles classés par catégorie suivant l'action du bloc.

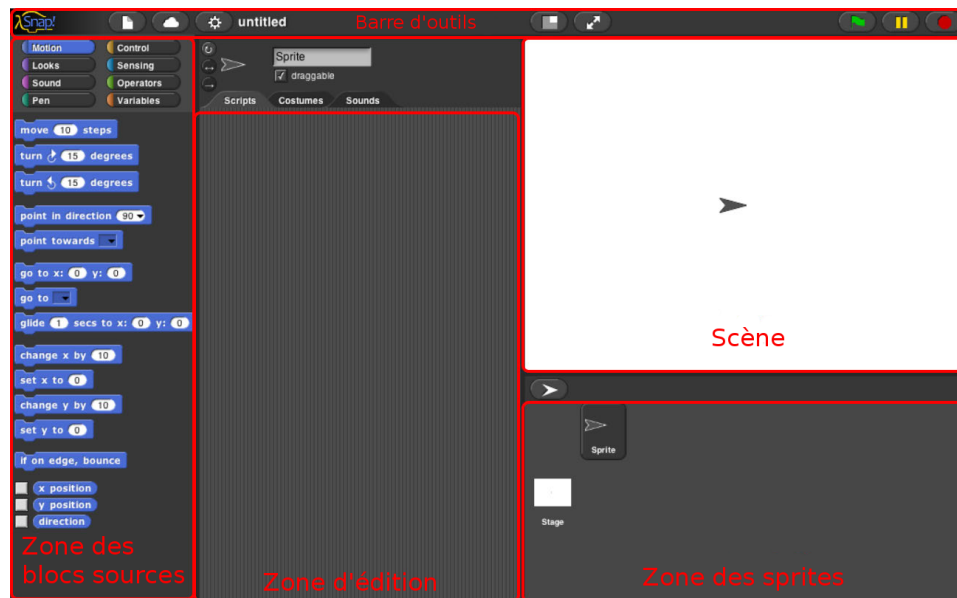


FIGURE 2.2 – Délimitation des zones de l'interface Snap!

Monde Une fois le programme fini, le drapeau vert dans la *barre d'outils*, lance le programme. Quand le programme est lancé, les actions des scripts sont visibles dans la *scène*. Cette zone représente le monde dans lequel vont évoluer les personnages. On peut y mettre un fond comme arrière-plan ou simplement le laisser blanc comme dans l'exemple, ceci se fait via le bouton stage de la *zone de sprites*.

2.2.2 Blocs

Il existe plusieurs types de blocs qui constituent un programme Snap! [3]. Sur la figure 2.3, le programme présente les différents types de blocs disponibles.

Commande

Le type principal de blocs est **commande**. Ces blocs peuvent être compris comme étant des procédures. En effet, ils exécutent une ou plusieurs opérations sur le système en fonction de paramètres fournis. Ces différents blocs doivent se baser soit sur une implémentation JavaScript pour les **commande**'s élémentaires, soit être une composition de **commande**'s pour fournir une nouvelle **commande** plus complexe ou abstraite.

Dans l'exemple de la figure 2.3, tous les blocs **commande** ont la forme d'une pièce de puzzle. On peut y voir qu'une succession de **commande**'s crée



FIGURE 2.3 – Exemple de programme Snap !

un script. Les **commande's** ont plusieurs couleurs suivant la catégorie à laquelle elles appartiennent : mouvement (bleu), apparence (mauve), contrôle (jaune), variable (orange), ...

Reporter

Les **reporter's** sont des fonctions. En effet, ils renvoient une valeur. Ils sont toujours utilisés en temps que paramètres d'un autre bloc. La plupart des **reporter's** sont des accesseurs à des variables ou à des états du système (position souris, heure ...).

Les **reporter's** sont des blocs de forme arrondie. La figure 2.3 montre différentes utilisations de **reporter** : somme, valeur aléatoire, position, etc. Tout comme pour les **commande's**, les **reporter's** peuvent être de diverses couleurs suivant leur catégorie.

Prédicat

Les **predicat's** sont des reporters qui retournent une valeur booléenne. Ils sont donc utilisés en conjonction avec des commandes demandant une condition. Les **predicat's** ont une forme hexagonale.

Chapeau

Les **hat's** sont des commandes spéciales, car ils sont le point d'entrée obligatoire d'un script en permettant de démarrer l'exécution de celui-ci. Plusieurs types d'événements peuvent lancer un script : une touche pressée, un clic sur le drapeau vert etc.

Dans la figure 2.3, le démarrage du programme est l'événement qui lance le script. Il est symbolisé par un drapeau vert.

2.2.3 Programme

Snap! est plus qu'une interface graphique permettant de construire un programme. L'analyse de son fonctionnement interne est l'objet de ce paragraphe.

Un programme est constitué de plusieurs processus. Chaque processus est exécuté en parallèle grâce à un ordonnanceur.

Un processus **Process** représente l'exécution d'un script, une pile de blocs. Il assure aussi le suivi de l'exécution du script : prochain bloc à exécuter, objet sur lequel il s'applique, contexte décrivant l'état courant, etc.

L'ordonnanceur **ThreadManager** appelle la fonction **runStep()** (extrait de code source 2.1) successivement sur chaque processus. Cette fonction exécute un certain nombre de blocs via **this.evaluateContext()** de manière atomique. Elle rend la main volontairement à l'ordonnanceur quand elle a fini. Comme il est possible d'écrire soi-même des blocs, **runStep()** rend aussi la main si trop de temps s'est écoulé depuis le début de l'exécution de cette étape. La convention veut que les processus rendent la main à la fin de chaque itération de boucle et quand une opération relative au temps ou synchrone est exécutée.

2.2.4 Interface graphique

Un autre élément intéressant à analyser chez Snap! est sa façon d'afficher l'interface dans la balise HTML **canvas** (voir code source 2.2). Toutes les fonctionnalités nécessaires à afficher tous les éléments graphiques de Snap! découlent de l'implémentation que l'on retrouve dans **morphic.js**.

morphic.js fournit les abstractions pour redessiner des parties de l'interface et pour interagir avec l'utilisateur. Le **canvas** utilisé possède un **world**. Ce **world** est la racine de l'arbre composé de **morph** et leurs sous-**morph**. Chaque **morph** peut être déplacé, redimensionné via le code source ou les manipulations de l'utilisateur.

La fonction principale de **morphic.js** consiste à parcourir continuellement tous les éléments du **world** pour redessiner ceux qui ont été modifiés. Le **world** permet à l'ordonnanceur d'exécuter une étape entre chaque itération. Le code source 2.2 montre que le **world** est rafraîchi toutes les 50 millisecondes. La fonction **drawNew()** sert à dessiner un **morph**. Le code source 2.3 montre que cette fonction dessine son objet sur une image stockée dans l'objet. Cette image provient d'un **canvas** virtuel généré grâce à l'image du **morph** parent.

2.3 Ruby on Rails

Rails [28] est une plateforme de développement d'applications web basée sur le langage de programmation Ruby [16]. Cette partie présente la philo-

```

1 Process.prototype.runStep = function () {
2   /*
3    a step is an an uninterruptable 'atom', it can consist
4    of several contexts, even of several blocks
5   */
6   // allow pausing in between atomic steps:
7   if (this.isPaused) {
8     return this.pauseStep();
9   }
10  this.readyToYield = false;
11  while (!this.readyToYield
12        && this.context
13        && (this.isAtomic ? (Date.now() - this.lastYield <
14                             this.timeout) : true) ) {
15    // also allow pausing inside atomic steps - for PAUSE
16    // block primitive:
17    if (this.isPaused) {
18      return this.pauseStep();
19    }
20    this.evaluateContext();
21  }
22  this.lastYield = Date.now();
23
24  // make sure to redraw atomic things
25  if (this.isAtomic &&
26      this.homeContext.receiver &&
27      this.homeContext.receiver.endWarp) {
28    this.homeContext.receiver.endWarp();
29    this.homeContext.receiver.startWarp();
30  }
31
32  if (this.readyToTerminate) {
33    while (this.context) {
34      this.popContext();
35    }
36    // pen optimization
37    if (this.homeContext.receiver &&
38        this.homeContext.receiver.endWarp) {
39      this.homeContext.receiver.endWarp();
40    }
41  }
42 };

```

Code source 2.1 – Fonction runStep() de Process

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Morphic!</title>
5     <script type="text/javascript" src="morphic.js">
6       </script>
7     <script type="text/javascript">
8       var world;
9
10      window.onload = function () {
11        world = new WorldMorph(
12          document.getElementById('world'));
13        setInterval(loop, 50);
14      };
15
16      function loop() {
17        world.doOneCycle();
18      }
19    </script>
20  </head>
21  <body>
22    <canvas id="world" tabindex="1" width="800" height="600" />
23  </body>
24 </html>
```

Code source 2.2 – Exemple d'utilisation de `morphic.js`

```
1 MyMorph.prototype.drawNew = function() {
2   var context;
3   this.image = newCanvas(this.extent());
4   context = this.image.getContext('2d');
5   // use context to paint stuff here
6 };
```

Code source 2.3 – Modèle pour la fonction `drawNew()`

sophie, l'architecture et des environnements de tests de Rails.

2.3.1 Philosophie

Rails part de l'hypothèse qu'il existe une meilleure façon d'aborder la création d'applications web [18]. Si le programmeur respecte ce "Rails way", il améliorera sa productivité et écrira moins de code. D'après Rails, s'il persiste à utiliser ses anciennes habitudes, le développeur s'amusera beaucoup moins en créant son application.

Pour atteindre cet objectif, Rails utilise deux principes majeurs :

Convention plutôt que configuration (CoC) [49] : Pour permettre au programmeur d'écrire moins de code, Rails permet de n'écrire que ce qui ne correspond pas aux conventions. Rails utilise la métaprogrammation pour fournir les conventions à tous les objets.

Ne vous répétez pas (DRY) [50] : Rails recommande une architecture où il faut mettre l'information à un endroit unique et bien déterminé. Ceci permet d'avoir un code plus court, plus maintenable, plus extensible et avec moins de bogues.

Un service aidant à mieux comprendre et respecter le "Rails way" est `Rails_best_practices`. Il fournit des métriques utiles pour détecter des écarts à la philosophie Rails. Cet outil s'utilise avec les conseils fournis par <http://rails-bestpractices.com> aidant à refactorer les morceaux de code qui ne respecteraient pas les conventions.

2.3.2 Architecture

Rails se base sur une architecture MVC [52] (figure 2.4).

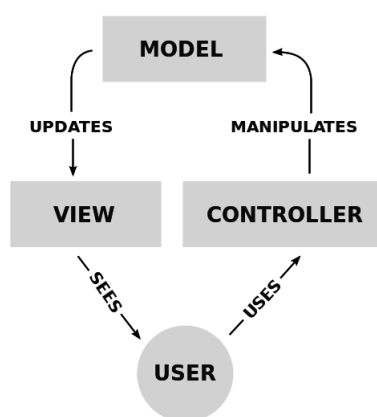


FIGURE 2.4 – Schéma interaction d'un modèle MVC

Modèle

Un modèle est typiquement une classe qui représente une table de la base de données. Une classe du modèle fournit aussi toutes les méthodes nécessaires à représenter et modifier l'objet dans le domaine d'application.

La correspondance entre un objet Ruby et la base de données est réalisée grâce à *Active Record*. Ce module de Rails permet de faire des appels sur des objets Ruby alors que dans d'autres framework, il faudrait utiliser des requêtes *SQL*.

Contrôleur

Le contrôleur a pour fonction d'accéder à une ressource. Une ressource est un modèle ou tout autre objet indirect tel qu'enregistrement, login, page d'accueil. . .

Il détermine quelle vue doit s'afficher et avec quels paramètres. Le contrôleur a donc la tâche de vérifier la sécurité et l'intégrité des données fournies par l'utilisateur. Ensuite, il peut interroger différents modèles et fournir les réponses à la vue adéquate.

Rails encourage à avoir des ressources *representational state transfer* (REST) [54], c'est-à-dire, des ressources avec une représentation unique et des actions unifiées telles que *create*, *new*, *edit*, *update*, *destroy*, *show*, *index*. Les actions disponibles sur un contrôleur sont déterminées par le fichier de configuration des routes.

Vue

Les vues correspondent à ce que les utilisateurs reçoivent et voient. Ce sont typiquement des pages HTML, mais aussi des PDF, objets JSON, fichiers, etc

Pour faciliter la création des vues, Rails permet d'utiliser des gems. Il existe notamment :

Haml [43] : un langage fournissant une syntaxe raccourcie du HTML. Il se base sur l'indentation plutôt que sur une syntaxe XML. Haml permet donc de respecter le principe **DRY** et d'améliorer la lisibilité par un code source plus court et bien indenté ;

jQuery [34] : cette bibliothèque JavaScript bien connue permet de modifier facilement le document courant, de gérer des événements, de créer des animations, etc. ;

CoffeeScript [6] : est un langage qui se compile en JavaScript. Il permet d'avoir une syntaxe plus claire, plus courte et d'ajouter des sucres syntaxiques par rapport à JavaScript ;

Bootstrap [19] : est un framework CSS qui découple le fond et la forme de l’affichage d’une page HTML. Ce framework donne un design adaptatif pour tous types d’écrans aux pages qui l’utilisent. Il fournit aussi de nombreux composants utiles : boutons, alertes, barres de progression, messages d’aide, etc.

2.3.3 Gems

Ruby fournit une manière de partager des fonctionnalités pour des programmes Ruby via les gems [38]. Rails lui-même est un gem et dépend de nombreux autres notamment Active Record présenté plus haut.

La communauté de Rails a écrit de nombreux gems permettant de rajouter facilement des fonctionnalités à une application. Le fichier **Gemfile** sert à lister les gems utilisés dans un projet. Le gestionnaire du projet doit faire attention à toutes ces dépendances pour garder son logiciel à jour. Pour ce faire, Gemnasium analyse le fichier **Gemfile** pour indiquer quels gems doivent être mis à jour. Gemnasium informe donc les gestionnaires d’un logiciel lorsqu’une faille de sécurité est découverte dans un gem et qu’il faut donc le mettre à jour.

Quelque exemples de gems intéressants sont présentés ci-après.

Paperclip [45] : permet de facilement récupérer et stocker des fichiers. Paperclip intègre donc la gestion de fichiers tiers de manière élégante dans une classe du modèle. Il fournit notamment divers pilotes pour stocker ces fichiers aussi bien en local que sur le nuage d’Amazon ou de Dropbox.

Devise [37] : gère la problématique de l’authentification des utilisateurs.

Rolify [22] : donne des rôles aux utilisateurs de manière générale (ex. administrateur) ou sur une ressource (ex. modérateur d’un forum).

Authority [36] : gère les droits des utilisateurs sur les différentes ressources des contrôleurs.

2.3.4 Tests

Il existe tout un écosystème autour de Rails pour fournir du code de qualité. Cette partie présente quelques outils utilisés dans les tests. Les outils les plus employés sont présentés ci-dessous, bien que ce ne soit pas les seuls existants.

Tests comportementaux

Un outil de test fort apprécié des programmeurs Ruby et plus particulièrement Rails est Cucumber [29]. Ce dernier réalise des tests dans un "behavior-driven development" [47] style. Ce style de tests se compose de

deux parties. D'une part, le client ou chef de projet écrit des scénarii qui décrivent l'utilisation de fonctionnalités du domaine en langage naturel (Gherkin [20]). D'autre part, le programmeur implémente ces scénarii. Ce type de test met l'accent sur ce que voit le client. Si tout ce que voit le client fonctionne, c'est que le coeur utile de l'application est correct.

Dans le cas de Rails, il est intéressant d'utiliser conjointement Cucumber et Capybara [32]. Ce dernier permet de contrôler un navigateur et donc de réaliser des tests à la place d'un utilisateur. Ces tests permettront d'assurer le bon fonctionnement de ce que souhaite l'utilisateur.

Intégration continue

Dans le cadre d'un développement actif, il est intéressant d'avoir un serveur qui réalise des tests d'intégration continue [48]. En effectuant les tests lors de chaque nouveau `commit`, les développeurs sont informés directement si une modification du code source a induit une régression des fonctionnalités. Ce service est proposé, entre autres, par Travis CI[26].

Travaux associés

Cette partie donne un aperçu de ce qui existe en matière d'apprentissage de la programmation.

Premièrement, il y aura un tour d'horizon des politiques gouvernementales de pays qui ont adopté l'apprentissage de la programmation dans leur programme officiel. Ceci présentera les avancées de cette problématique et aidera à envisager différentes manières de la traiter.

Ensuite, l'attention sera portée sur des organisations non gouvernementales qui ont pour but de promouvoir l'enseignement de la programmation.

La troisième partie décrira les langages utilisés dans les organisations présentées précédemment.

Enfin, des concepts différenciateurs seront extraits de chaque initiative afin les distinguer l'une de l'autre.

3.1 État de la programmation dans le monde

Cette section fait un tour d'horizon de différents pays qui enseignent la programmation aux jeunes. L'accent est mis spécialement sur leur positionnement politique et leur manière d'intégrer l'apprentissage de la programmation. Ces pays ont été choisis sur base du rapport de l'académie des sciences de France [21] et de la Royal Society [25], car ils apportaient différentes conceptions et mise en application de l'apprentissage de la programmation.

3.1.1 Angleterre

L'apprentissage de l'informatique en Angleterre [23] n'est pas nouveau. Pendant longtemps, il était centré sur les technologies de l'information et de la communication (TIC). C'est donc principalement l'utilisation de l'informatique, et non sa conception qui était enseignée.

En 2010, une étude a été commandée à la Royal Society pour évaluer cet apprentissage. Un an plus tard, le rapport a révélé que l'enseignement n'était ni efficace ni en adéquation avec l'évolution de l'informatique. La Royal Society a suggéré d'adapter les matières abordées en informatique en enseignant l'apprentissage de la programmation. Sur base de ce rapport, les programmes de cours ont été modifiés en 2012.

3.1.2 France

En France [51] [46], depuis deux ans, l'informatique fait partie intégrante du programme du baccalauréat de type S. Une des matières dispensées est "Informatique et sciences du numérique". Cette matière se subdivise en quatre sous-parties : représentation de l'information, algorithmique, langages de programmation et architectures matérielles. Cette approche est donc également basée sur l'apprentissage de la programmation plutôt que sur les TIC.

3.1.3 Corée du Sud

La Corée du Sud enseigne l'informatique depuis longtemps et à tous les niveaux de l'enseignement. La culture numérique dans les pays asiatiques est fort différente de ce que l'on connaît chez nous. Par exemple, une carrière dans le jeu vidéo est quelque chose de tout à fait normal. L'informatique est vraiment omniprésente dans cette culture, il est donc normal que son apprentissage commence dès l'enseignement fondamental.

3.1.4 Grèce

L'apprentissage des sciences informatiques prend une place importante dans les programmes grecs. Dès 6 ans, les enfants sont confrontés à l'informatique à l'école. À cet âge, ils apprennent principalement la maîtrise de l'outil. Dès 10 ans, leurs cours d'informatique prennent une tournure plus algorithmique et donc plus proche des sciences informatiques.

3.1.5 Nouvelle-Zélande

Ce pays a adopté récemment les sciences informatiques dans son programme d'étude. Les cours sont dispensés à partir de 15 ans. Ils comprennent l'apprentissage de la programmation et de concepts informatiques en général. La Nouvelle-Zélande, en introduisant les cours tardivement, s'inscrit dans une logique similaire à la France sans toutefois restreindre l'informatique aux options scientifiques.

3.1.6 Belgique

L'apprentissage de la programmation est déjà bien avancé dans plusieurs pays. Ce n'est malheureusement pas encore tout à fait le cas en Belgique. En effet, quelques initiatives locales existent, mais aucune décision politique n'a été prise jusqu'à présent.

3.2 Organisations existantes

Il sera présenté ici un ensemble d'initiatives à ce travail qui s'inscrivent la logique de ce travail. Elles sont toutes non gouvernementales et proposent d'enseigner la programmation de différentes manières.

3.2.1 Code.org

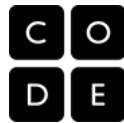


FIGURE 3.1 – Logo de code.org

Code.org [12] est une organisation sans but lucratif des USA qui a pour objectifs :

- apporter l'informatique dans toutes les classes de secondaire aux États-Unis ;
- démontrer le succès de l'utilisation de cours en ligne dans l'enseignement public ;
- ajouter l'informatique dans les bases des programmes de sciences et de mathématique des 50 états ;
- employer la connaissance technique collective pour améliorer l'apprentissage de l'informatique dans le monde ;
- augmenter la représentation féminine et de personnes de couleurs dans l'informatique.

Pour ce faire, Code.org fournit une plate-forme [14] web qui permet aux professeurs de suivre leurs élèves grâce à un système de classes. Ce programme d'apprentissage se base sur Blockly (voir 3.3.1).

Toutes les ressources sont gratuites et librement utilisables [13]. Elles sont conçues pour que les professeurs comme les élèves puissent commencer le cours sans connaître l'informatique (une assistance gratuite est proposée au professeur si nécessaire).

Le site propose aux professeurs de se faire récompenser s'ils arrivent à finir les 27 missions proposées avec minimum 15 élèves. Dans ce cas, ils gagnent 750\$. S'ils ont au moins 7 filles dans le groupe, ils peuvent prétendre à 250\$ supplémentaires.

Déroulement des leçons

Code.org propose des sessions d'une heure de travail/jeu/apprentissage. Chaque unité est découpée en missions courtes (ex :5-20) apportant un

concept de programmation. Avant chaque concept, une petite vidéo l'introduit et donne des exemples d'utilisation.

Il propose de faire travailler les élèves en binôme [53], ce qui entraîne moins de questions au professeur et permet de mieux s'approprier la matière. Le travail par binôme casse également l'image du "geek" et montre que la programmation est une science sociale et collaborative. De plus, moins d'ordinateurs sont nécessaires.

Le site explique également que pour faire participer tous les élèves, il faut avoir confiance en leurs compétences. La philosophie est d'inciter les premiers groupes à aider les derniers.

Quand un élève a une question, Code.org recommande de la soumettre à 3 de ses camarades avant de la poser au professeur. Cela permet aussi d'éviter les questions de distraction ou de manque de compréhension.

Pour chaque petite mission, un test automatisé informe si la mission est réussie ou non. Si celle-ci est réussie, la plateforme propose la mission suivante. Dans les premières missions, il y a également un compteur de blocs qui informe du nombre de blocs nécessaires pour réaliser la mission de manière optimale.

3.2.2 CoderDojo



FIGURE 3.2 – Logo de CoderDojo

CoderDojo [15] est un réseau open source de Clubs de programmation dans le sens le plus large du terme. Tous les dojos sont donc autonomes. Des enfants de 5 à 17 ans y apprennent la programmation (site web, application, jeux...). La seule règle est "Above All : Be Cool" qui est mise en pratique en créant simplement des espaces d'échanges de savoirs amicaux et sociables.

CoderDojo a été créé par James Whelton, un irlandais de 18 ans, et Bill Liao, un entrepreneur australien à Cork. James, après avoir hacké l'iPod nano, a eu des demandes de jeunes enfants pour avoir des cours de programmation. Beaucoup de gens de Dublin ont été à ses cours et donc un nouveau Dojo y a été créé. Ensuite, cela s'est étendu à tout le globe.

3.2.3 Code Club



FIGURE 3.3 – Logo de Code Club

Code Club[11] est un réseau national de Clubs mené par des bénévoles en dehors des heures de cours. Leurs activités s’adressent à des enfants de 9 à 11 ans.

Ils créent le matériel pour permettre à des bénévoles de donner des cours parascolaires d’environ une heure par semaine. Ils proposent d’utiliser dans cet ordre scratch, HTML/css et puis Python. Ils ont pour objectif que les 21000 écoles fondamentales anglaises aient leur club.

Leur philosophie est de favoriser l’amusement, la créativité et l’exploration avant l’apprentissage des concepts de programmation.

3.2.4 KidsCode



FIGURE 3.4 – Logo de KidsCode

KidsCode [35] est une petite initiative wallonne qui organise actuellement un stage pour les enfants de 10 à 14 ans. Les ateliers sont dispensés par deux informaticiens qui proposent de partager leur passion. Ils encadrent les jeunes de manière ludique dans le but de les rendre autonomes.

KidsCode propose d’apprendre la programmation grâce au langage Python. Ce langage a été choisi pour sa facilité tout comme pour son utilisation en condition réelle.

KidsCode a été conçu et pensé en association avec l’incubateur de startup Nestup. Cette initiative est le fruit du constat par les entreprises, d’un manque d’intérêt pour la programmation chez les plus jeunes. Ceci entraîne un manque de vocations dans les startups.

3.3 Langages de programmation

Cette partie traite des différents éditeurs et langages de programmation utilisés dans les organisations présentées au point précédent.

3.3.1 Blockly



FIGURE 3.5 – Logo de Blockly

Blockly [8] est un langage de programmation graphique basé sur des technologies du web.

Blockly a comme particularité :

- de s'exécuter dans un navigateur ;
- d'exporter du code source en JavaScript, Dart, etc. ;
- d'être open source ;
- d'être haut niveau.

Il n'est pas directement une plateforme d'éducation dans le sens où il peut être utilisé autant pour l'éducation que pour le business, les jeux, etc. En effet, Blockly est juste une interface graphique en pièces de puzzle sur des fonctions écrites en JavaScript. Celles-ci permettent d'interagir avec le domaine d'utilisation. Il faut donc que pour chaque domaine, une implémentation des blocs soit réalisée.

Blockly a été conçu avec certaines propriétés choisies lors de sa création. Les trois premières augmentent la compréhension des néophytes, les autres portent sur des facilités du langage. Les propriétés décidées lors de la conception du langage sont [9] :

- des indices de listes commençant à 1 ;
- des noms de variables non sensibles à la casse ;
- pas de portée de variables, elles sont toutes globales ;
- la possibilité de faire un export en JavaScript ;
- un code natif généré proche de celui des blocs.

3.3.2 Scratch

Scratch [27] est un langage de programmation graphique développé par le MIT pour apprendre aux enfants la programmation. C'est l'interface qui permet de faire des scripts facilement grâce à une programmation par blocs et du glisser-déposer.



FIGURE 3.6 – Logo de Scratch

Il a été pensé pour être un outil créatif pour réaliser des histoires, des jeux, des simulations, de l'art, etc. Il a, par exemple, son propre éditeur d'image et de son. Un autre but de ce langage est d'être simple à utiliser et à apprendre. Il a en effet été conçu pour des enfants n'ayant aucune connaissance préalable en programmation.

Actuellement Scratch est à sa version 2 qui est une version web. Cette version a été complètement réécrite en flash par rapport à la version 1 en Smalltalk. De plus, la version 2 n'est plus open source contrairement à la première version.

Comme le montre la figure 3.7, l'interface de Scratch se divise en plusieurs grandes parties :

1. sur la gauche, il y a la zone de dessin dans laquelle s'anime les composants graphiques des scripts ;
2. au milieu, une liste des blocs disponibles triés par catégorie ;
3. sur la droite, la zone de script qui contient tous les scripts liés au sprite sélectionné.

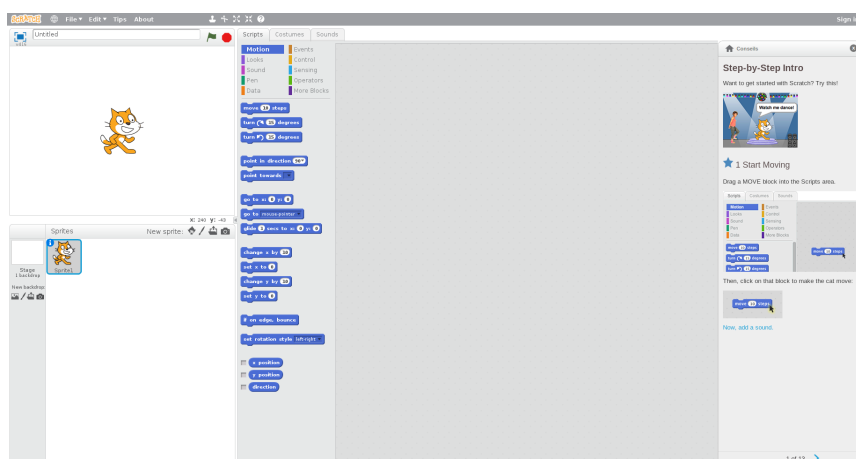


FIGURE 3.7 – Interface de Scratch

3.3.3 Snap !



FIGURE 3.8 – Logo de Snap !

Snap! [41] est un langage de programmation de "glisser-déposer" de blocs. C'est une ré-implémentation et une extension du langage Scratch du MIT. Il a été pensé et conçu pour être orienté web. Il est donc implémenté en JavaScript.

Ce langage est né en 2011 et a été créé par Jens Möning, docteur de l'université de Berkeley. Il se distingue de son père Scratch par l'ajout de :

1. fonctions et procédures de première classe. Une entité dite de première classe est une entité qui supporte toutes les opérations disponibles sur d'autres entités. Un exemple est passer une fonction en paramètre ;
2. listes de première classe. Un exemple est une liste de liste ;
3. sprite de première classe. Un exemple est la composition de sprite, par exemple personnage est représenté par un sprite mais ses bras sont des sous-sprites qui peuvent soit se déplacer avec l'ensemble du corps soit de manière indépendante.

3.3.4 Python



FIGURE 3.9 – Logo de Python

Python [24] est un langage de programmation qu'il ne faut plus présenter. Il a beaucoup d'avantages dont celui d'avoir une syntaxe légère et d'être facile à prendre en main. Une grande communauté et beaucoup de bibliothèques, dont la fameuse `turtle`, en fait un excellent langage pour démarrer dans la programmation.

Cependant, devoir apprendre un langage de programmation et la logique informatique en même temps complique la tâche. De plus, lorsqu'on commence un nouveau langage de programmation, il faut également apprendre ses bonnes pratiques de codage.

C'est donc un langage qui ne convient pas aux trop jeunes ni à ceux qui ne sont pas vraiment investis dans l'apprentissage de la programmation.

3.4 Concepts différenciateurs

Ce chapitre va mettre en lumière les différences et similitudes des méthodes d'apprentissage de la programmation en utilisant une série de critères tels que le public cible, les moyens, le lieu et les personnes.

3.4.1 Pour qui ?

L'apprentissage de la programmation aux enfants est différencié sur base de plusieurs critères tels que l'âge, l'origine, la classe sociale, le genre, etc. Les organisations auront donc soit un public visé restreint soit différents programmes pour les groupes d'enfants.

Âge

La méthode d'apprentissage doit être différente suivant l'âge. Les enfants n'ont pas tous la maturité pour apprendre des concepts abstraits (boucle, parallélisation...). Par exemple, pour des élèves de maternelle, un tableau interactif sera un grand plus, car ils n'ont pas encore la dextérité souhaitée pour manipuler la souris. Et au contraire, des enfants plus grands seront plus autonomes et pourront travailler par binôme voir seul.

Genre et origine

Certaines organisations, telle Code.org, mettent l'accent sur l'accessibilité pour tous en incitant à la participation des filles qui sont sous-représentées en informatique.

Les personnes qui ne peuvent acquérir un ordinateur ou se payer un stage ne doivent pas être délaissées.

3.4.2 Comment ?

Les principales différences entre toutes les organisations se situent dans la manière d'aborder l'apprentissage. Elles peuvent être de plusieurs ordres, tels que : les outils utilisés, les concepts abordés et l'environnement de travail.

Outils

Un aspect pratique qui différencie les apprentissages est le type de langage utilisé. Ici encore différentes écoles s'affrontent.

Le langage réel Avec ce type de langage, les enfants apprennent la programmation et le langage utilisé. Ces langages sont plus difficiles à prendre en main, mais permettent plus d'applications concrètes.

Le langage visuel Dans la programmation, le plus important est la logique. Un langage qui se rapproche des diagrammes allège la syntaxe, par exemple, avec un langage de programmation par blocs.

Le langage web Dans le monde d'aujourd'hui, le web est partout. L'accent doit donc être porté sur l'apprentissage de ce que les enfants voient au quotidien.

Concepts

Tous les cours se divisent en deux parties : la théorie et la pratique. Il existe de multiples manières d'assembler les deux dans un cours unique. L'ordre dans lequel les concepts de programmation sont abordés peut se différencier sur plusieurs points :

- certains cours commencent par la théorie et demandent ensuite de l'appliquer ;
- d'autres proposent des exercices avant d'expliquer ce que les enfants ont utilisé de manière intuitive ;
- d'autres encore découpent la matière en petits sous-ensembles avant d'appliquer la première ou la seconde technique.

Ces différentes façons d'aborder l'apprentissage mènent à diverses conceptions de missions. Celles-ci peuvent être très denses ou au contraire une succession de missions simples. Les missions denses sont souvent constituées de nombreux concepts de programmation (boucle, condition, fonction...) pour avoir directement un objectif concret alors que les missions simples essaient de bien séparer l'apprentissage pour fournir une étude la plus progressive possible.

Dans certains pays, la programmation est associée aux cours de sciences. Elle devient alors un outil qui perd du même coup une partie de son côté ludique. Par contre, elle touche un plus grand nombre d'enfants.

Environnement de travail

Les leçons peuvent être organisées de différentes manières :

En groupe L'apprentissage en groupe est souvent réalisé avec un tableau interactif ou un projecteur [33] ;

Par binôme La programmation en binôme est la plus répandue. Faire travailler les enfants par deux les obligent à interagir entre eux et à essayer de résoudre eux-mêmes les problèmes qu'ils rencontrent. Cela permet aussi de diminuer le nombre d'ordinateurs utilisés ;

Individuel Cette technique est très peu utilisée, car elle demande beaucoup de moyens matériels et personnels. De plus, lors d'un apprentissage individuel, les élèves se retrouvent vite coincés et demandent de l'aide ;

A domicile De nombreuses plateformes proposent des cours en ligne qui peuvent être suivis au rythme de chacun. Néanmoins, pour avoir de l'aide, il faut se tourner vers une personne compétente ou vers les forums. Ces formations sont souvent plus théoriques et demandent plus de lecture, ce qui empêche les jeunes enfants de les suivre.

3.4.3 De qui ?

Chaque initiative s'organise suivant différents critères tels que l'intégration ou non dans un programme scolaire, les compétences des encadrants et le système de gestion des ressources

Type d'organisation

Les cours d'informatique sont soit donnés dans le cadre d'un programme officiel soit en parascolaire.

Dans le premier cas, les élèves ne viennent pas tous par choix. Ceci peut entraîner entre autres des blocages et des réticences à participer correctement à l'activité.

Pour les cours donnés en parascolaire, des petites structures très locales se partagent le terrain avec de grandes organisations à visées internationales. Le coût des activités et les horaires en sont les problèmes typiques.

Encadrants

La plupart des cours sont donnés par des informaticiens ou au minimum des personnes formées à l'informatique. Mais comme le nombre d'informaticiens est réduit, certaines organisations proposent des cours clés en main pour des encadrants sans connaissance particulière en programmation. Ces professeurs apprennent en même temps que leurs élèves et les aident à réfléchir.

Le type de rémunération des encadrants est assez diversifié. Certains reçoivent des primes en fonction de la réussite de leur classe, d'autres sont payés par l'état ou directement par les élèves, d'autres encore sont bénévoles.

Gestion des ressources

Suivant l'organisation, les cours sont plus ou moins figés. Certaines permettent à leur communauté de créer et/ou de modifier des cours tandis que d'autres ont des équipes dédiées à cette tâche.

Définition de la problématique

Ce chapitre a pour but de situer Rsnap par rapport aux initiatives qui s'inscrivent dans la logique de ce travail. Premièrement, Rsnap est positionné en reprenant les concepts différenciateurs du chapitre précédent 3.4. Ensuite, les besoins que la plateforme doit remplir sont abordés. La troisième partie traite des choix technologiques qui ont été pris dans le cadre de ce travail.

4.1 Positionnement de Rsnap

Rsnap est la solution développée dans le cadre de ce travail. Elle est conçue pour être l'outil privilégié pour faire interagir les professeurs avec leurs élèves dans l'apprentissage de la programmation.

Cette première partie du chapitre présente qui, comment et grâce à qui Rsnap est utilisé.

4.1.1 Âge, genre et origine des utilisateurs

Le positionnement Rsnap vis-à-vis du public qu'elle vise est présenté dans cette section.

Âge Rsnap se veut une application qui vise un public de 10 à 14 ans. Les missions qui ont été développées ciblent particulièrement cette tranche d'âge. Cette question est nuancée par l'analyse des résultats des expérimentations abordée dans le chapitre 6.2.2. Toutefois, l'application permettant la création de missions de manière autonome, d'autres tranches d'âge peuvent s'y appliquer. La créativité des utilisateurs est la seule limite formelle.

Origine et genre En ce qui concerne les particularités des utilisateurs, aucune discrimination dans quelque sens que ce soit n'a été faite lors de la conception de ce travail, tant pour attirer un public que pour l'exclure.

Comme l'idée est de proposer l'application dans les écoles de la Fédération Wallonie-Bruxelles, elle a été pensée pour un public francophone.

4.1.2 Outils, concepts et environnement de travail

Cette section explique comment sont utilisées les différentes ressources pour apprendre la programmation aux enfants.

Outils L'apprentissage de la programmation dans le but d'améliorer l'esprit logique des jeunes est un des objectifs principaux de ce travail. Dans cette optique, Rsnap se base sur un langage visuel (section 3.3). L'accent est mis sur l'acquisition d'un esprit logique grâce à des exercices de logiques plutôt que sur l'apprentissage d'un langage de programmation. En effet, suivant l'adage "il faut diviser pour mieux régner", vouloir tout faire en même temps ne convient pas à tous. En se concentrant sur la logique, cela assure une meilleure acquisition de la matière. De plus, un des buts de l'application Rsnap est aussi de susciter des vocations en programmation, ce qui paraît en découler assez naturellement.

En outre, un des objectifs de ce travail vise à ce que les personnes qui guident l'activité n'aient pas besoin de connaissances particulières en programmation.

Un vrai langage de programmation a aussi été écarté, car les encadrants auraient dû au minimum avoir connaissance de sa syntaxe.

Concepts La manière dont les concepts sont abordés dans Rsnap est envisagée par la procédure suivante :

- une vidéo d'introduction à la mission ;
- un texte descriptif de la mission qui reprend les concepts théoriques introduits dans celle-ci ;
- une fois dans le programme, les jeunes n'ont plus d'explication explicite, ce qui les incite à travailler avec leur intuition tout en pouvant, si nécessaire, récupérer la description du point 2 ;
- une page d'aide est disponible pour chaque bloc dans le menu contextuel.

Les missions implémentées sont de petites missions introduisant un à deux grands concepts à la fois. Ces petites missions ont pour but d'être assemblées pour faire un programme final plus important.

Plus d'informations à propos du découpage et du contenu des missions est disponible dans la section 5.1.

Environnement de ce travail L'environnement de ce travail est déterminé prioritairement par son milieu d'utilisation, à savoir les écoles.

L'indépendance des jeunes par rapport au référent oriente l'activité vers la formation de binôme. Toutefois, comme le groupe animé est une classe, une introduction collective est possible et souhaitable. Si les jeunes le souhaitent, ils peuvent également avoir accès au site web en dehors du cadre scolaire.

4.1.3 Type d'organisation, enseignants, création des cours

Cette dernière section explique comment se positionne Rsnap par rapport aux personnes qui créeront les missions.

Type d'organisation Il n'y a pas d'organisation officielle qui supporte Rsnap. Elle pourrait être soit reprise par le ministère de l'enseignement, soit être indépendante, mais travailler pour ce dernier.

Enseignants En ce qui concerne les enseignants, le projet a été créé afin que la personne qui dispense l'activité n'ait pas besoin de connaissances spécifiques en programmation. Le simple fait de réaliser les projets avant les jeunes devrait être suffisant pour acquérir la logique nécessaire et savoir la transmettre.

Création des cours La création des cours est un point sur lequel Rsnap se distingue de beaucoup d'autres. En effet, une partie des missions existe déjà et est intégrée à ce travail. Les professeurs peuvent également créer des missions et les partager avec le reste de la communauté. Tout comme ils peuvent aussi reprendre des missions existantes et les améliorer ou les adapter.

4.2 Choix technologiques

Pour mettre en pratique les concepts du chapitre précédent, des critères d'utilisation ont dû être déterminés. La première partie a pour but de les présenter. Ensuite, les choix effectués à propos de la plateforme d'apprentissage sont développés, suivis par ceux concernant le langage d'apprentissage.

4.2.1 Critères d'utilisations

Suite au positionnement de Rsnap (section 4.1), une analyse des critères d'utilisation a été réalisée. Ces critères sont les suivants :

Apprentissage de la programmation pour fournir un environnement de développement intégré pour la pédagogie de la programmation.

Accessibilité au plus grand nombre et compréhensible pour le public visé.

Différenciation des utilisateurs afin de fournir une interface personnalisée suivant que l'utilisateur est un professeur ou un élève.

Indépendance du matériel afin d'avoir une plateforme accessible sur le plus d'ordinateurs et autres périphériques possibles.

Fiabilité et évolutivité pour avoir la possibilité de rajouter des fonctionnalités tout en gardant une stabilité de l'application.

Facilité à mettre en place et à maintenir en vue d'être déployable facilement dans une école si cette dernière a le matériel adéquat à savoir un serveur web.

Sauvegarde de l'avancement pour permettre à chaque utilisateur de savoir où en est la résolution de ses missions.

4.2.2 Plateforme d'apprentissage

La plateforme d'apprentissage est la partie de l'application qui permet d'aider à gérer un ensemble d'élèves en fournissant une interface pour mettre à disposition des missions et récupérer les soumissions.

Sur base des critères précités, les choix suivants ont été faits :

Technologie web Pour faciliter l'utilisation de la plateforme par tous, il est utile de choisir des technologies web. En effet, il suffit d'un navigateur et d'une connexion internet pour que les utilisateurs puissent y accéder.

Logiciel libre La connaissance ne devant pas être la propriété de quelques-uns, ce travail a visé à la rendre accessible à tous. Les logiciels libres forcent à respecter ce droit d'accès à l'enseignement et à la culture. De plus, cela permet à tout un chacun d'héberger l'infrastructure en interne.

MVC Pour séparer les différentes problématiques dans des composants bien définis, l'architecture modèle-vue-contrôleur (MVC) est un choix courant. Dans le cas de Rsnap, le modèle permet de gérer les concepts d'utilisateurs, d'exercices et de résolutions. La possibilité de donner des droits différenciés aux élèves ou aux professeurs se fait au niveau des contrôleurs. Les vues fournissent l'indépendance par rapport au matériel, car elles peuvent être spécialisées en fonction de celui-ci.

Un logiciel qui permet de satisfaire tous ces choix en même temps est Ruby on Rails (Rails). En effet, comme expliqué dans la section 2.3.3, Rails peut bénéficier de différents gems pour avoir facilement les fonctionnalités demandées. C'est aussi du fait de la connaissance approfondie de cet outil par les auteurs de ce travail que cette plateforme a été choisie. Ces deux éléments ont permis de développer rapidement et correctement une application qui correspond aux critères.

4.2.3 Langage d'apprentissage

Le langage d'apprentissage est utilisé dans l'application pour inculquer la programmation. En effet, c'est ce langage de programmation qu'utiliseront les enfants pour réaliser les missions.

Les critères d'utilisation ont déterminé certains choix :

Langage complet Pour pouvoir apprendre tous les concepts de programmation, il faut que le langage soit un langage complet dans le sens de Turing. De plus, un langage impératif est intéressant, car c'est le paradigme le plus répandu.

Blocs Les enfants ont plus de facilité à s'approprier la matière quand elle se présente de manière visuelle. Un langage par blocs est donc intéressant pour eux. Son interface colorée est attrayante. Son interface graphique fournit un retour direct des résultats de son programme à l'utilisateur. En effet, il peut voir évoluer son curseur dans la fenêtre dédiée.

JavaScript Pour que le langage soit accessible au plus grand nombre, il faut qu'il soit disponible sur la majorité des navigateurs actuels des ordinateurs et des tablettes. Le standard du web actuel pour faire de l'animation est le JavaScript.

Ces différents choix mènent à sélectionner Snap! BYOB (Snap!) et Blockly comme prétendant au langage d'apprentissage pour ce travail. Cependant, Blockly a le désavantage de ne pas être un vrai langage. Donc si les professeurs veulent créer des nouvelles missions, il y a de forte chance que des blocs leur manque. Ils devront donc soit avoir les compétence nécessaire en JavaScript pour les implémenter soit abandonner la création de la mission. Snap! n'a pas ces limitations et à donc été choisi pour ce travail.

Présentation de la plateforme Rsnap

Ce chapitre va présenter la plateforme Rsnap en commençant par les missions qui ont été créées, ensuite vient la présentation de l'interface de Snap! BYOB (Snap!) et les modifications qui y ont été apportées. Enfin, vient la présentation de la partie web de l'interface Rsnap.

5.1 Missions

Le moyen de présenter les concepts retenus est une approche par missions. Cette approche a été choisie sur base de l'analyse des autres initiatives similaires dans le chapitre 3. D'autres références présentant des cours avec Scratch ont aussi été utilisées tel que [40], [31], [10], [4] et [17]. Ces références ont été utilisées pour connaître le type d'exercices proposés aux enfants et s'en inspirer. Dans le cadre de ce travail, une série de missions ont été produites pour avoir une plateforme utilisable.

Après l'explication du schéma général des missions, une description de chacune des quatre missions est présentée ainsi que les concepts qu'elles apportent.

5.1.1 Schéma général

Une fois le choix de l'approche par mission fait, il a fallu définir comment amener la théorie de la programmation à travers ces missions. Il s'est avéré que pour capter l'attention et l'intérêt des élèves, il était nécessaire qu'elles aient un but final. Chaque mission réussie mène à la mission suivante et introduit de nouveaux concepts de programmation. La dernière mission intègre les différents concepts à travers un jeu de poursuite. Dans ce jeu, un chien doit courir après un chat et le manger. Cet objectif ludique permet d'introduire plusieurs concepts de programmation intéressants tels que la succession d'instructions, les conditions, les boucles, les événements etc.

Les missions ont donc été conçues dans le but de pouvoir réaliser ce jeu, ce qui met l'accent d'avantage sur la réalisation d'un jeu que sur l'apprentissage de concepts. Dans cette optique, les concepts nécessaires à la réalisation de la mission "Tu ne m'aterras pas" sont introduits par trois pré-missions. L'introduction progressive des concepts permet de se concentrer sur un à

deux grands concepts par mission. Ceci diminue les introductions théoriques des missions et augmente les temps pendant lesquels les enfants créent les programmes.

La chronologie des missions est pensée pour avoir une progression. La première introduit la programmation impérative, la seconde amène les boucles et la troisième donne l'intuition de la programmation événementielle. La dernière mission réutilise tous ces concepts.

Les trois pré-missions sont : "en voiture", "l'hélicoptère" et "soyons courtois".

5.1.2 En voiture

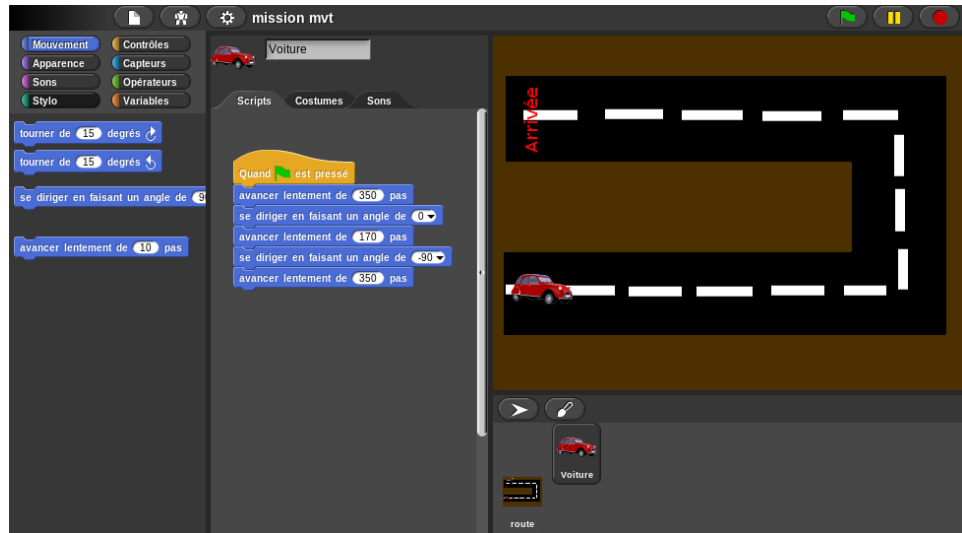


FIGURE 5.1 – Mission de la voiture

Dans cette première mission, voir figure 5.1, les participants sont mis dans un bolide qui doit atteindre la ligne d'arrivée en restant sur la route. Si la voiture sort de la route, elle explose. À chaque lancement du script la voiture reprend sa position de départ.

Cette mission vise à ce que les participants puissent prendre en main l'interface de Snap! et à introduire la notion de suite d'instructions. Ils doivent enchaîner des blocs de déplacement dans lesquelles, le nombre de pas et l'angle de virage sont à compléter. A la base les élèves n'ont rien en dessous des premiers blocs jaunes, La figure 5.1 montre la solution de la mission.

Le fait de devoir prévoir les instructions n'est pas un concept facile pour le public cible. Beaucoup exécutent une opération et puis cherchent à savoir comment continuer à partir de leur nouvelle position. Dans cette mission, la

voiture retournant à chaque fois à son point de départ, les élèves sont forcés à réfléchir de manière globale et anticipativement. Cette mission les introduit également à la programmation impérative.

Cette mission réussie, l'élève peut passer à la mission de l'hélicoptère.

5.1.3 L'hélicoptère

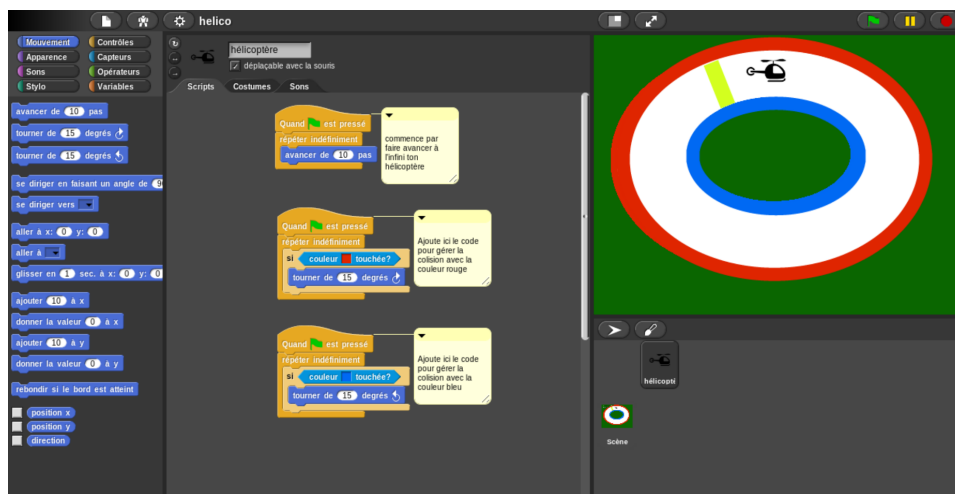


FIGURE 5.2 – Mission de l'hélicoptère

Pour cette mission, voir figure 5.2, les élèves sont aux commandes d'un hélicoptère. Leur but est de faire le tour de la piste. La piste est un ellipsoïde bordé d'herbe. Cette forme ellipsoïde les force à utiliser des boucles et non un grand nombre de blocs créant une solution particulière. Si l'hélicoptère touche l'herbe, il explose. Sur le contour extérieur de la piste, une bande rouge et une bande bleue sont dessinées. Le but de cette mission est de tourner dès qu'une de ces deux couleurs est touchée pour éviter l'explosion de l'hélicoptère. Comme pour la mission précédente, à chaque lancement de script, l'hélicoptère se repositionne à son point de départ. Au départ de la mission il n'y a que les blocs de tête qui sont présents.

Les concepts introduits par cette mission sont :

- les boucles, particulièrement le **boucler infiniment** ;
- la gestion des collisions grâce aux capteurs de couleurs ;
- la division en sous-processus (un pour avancer, un pour la collision rouge et un pour la bleue).

5.1.4 Soyons courtois

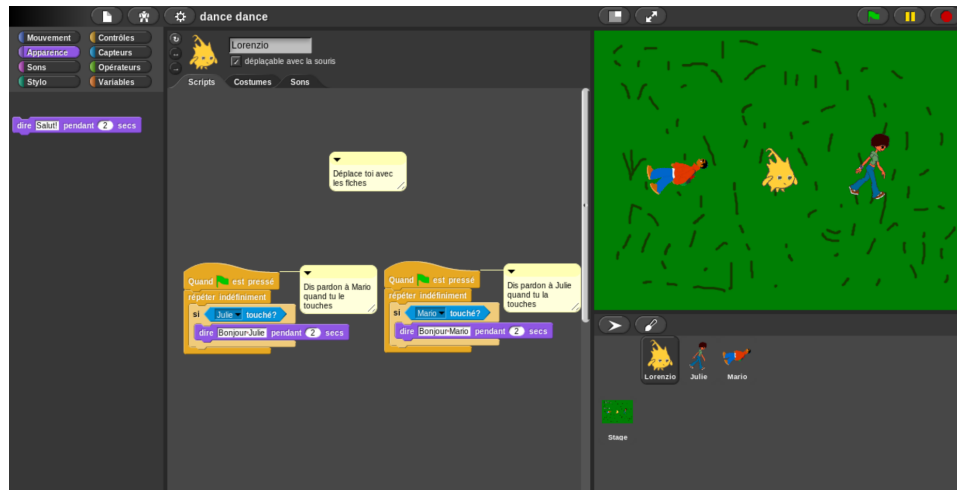


FIGURE 5.3 – Mission de soyons courtois

Dans cette dernière mission de préparation, voir figure 5.3, le participant dirige un personnage qui croise d'autres personnages. Ces derniers disent bonjour à chaque fois qu'ils croisent quelqu'un. Le but de la mission est que le personnage de l'élève réponde. Au départ de la mission il n'y a que les blocs de tête qui sont présents.

Les concepts introduits par cette mission sont :

- la gestion des collisions grâce au capteur sur leur personnage ;
- la division en sous-processus (un pour chaque personne) ;
- l'introduction à l'interactivité de l'interface, leur personnage est déplaçable à l'aide des flèches du clavier ;
- la gestion des dialogues et de l'affichage de texte.

5.1.5 Tu ne m'attraperas jamais

Cette mission est la dernière de la série et a pour but de mettre ensemble tous les concepts vus dans les missions de préparation. Dans cette mission, les élèves doivent faire en sorte d'avoir deux personnages et de pouvoir les déplacer séparément à l'aide des touches. Le chien doit courir après le chat. Sur base de propositions faites, les participants déterminent librement l'application exacte des effets de la poursuite. Par exemple, si le chat est touché par le chien, il crie. Aucun bloc n'est présent au départ de cette mission.

Cette mission reprend les concepts vus dans les missions de préparation, auxquels elle ajoute :

- modification de l'interface de jeu ;
- l'activation d'un script sur un événement (déplacements).

En ce qui concerne les déplacements, ils sont déjà présents dans la mission précédente, mais à l'état passif. Les élèves ne font que les utiliser. Dans cette mission, ils doivent coder eux-mêmes le fait de pouvoir déplacer leurs personnages à l'aide du clavier. Cette application approfondie du concept de déplacement fixe un concept important pour utiliser une interaction avec le clavier.

5.2 Snap !

Ce travail part d'un environnement de programmation existant : Snap!. Ce projet a pour but de fournir une interface et un environnement supportant la programmation par blocs. Cependant, il ne s'inscrit pas dans le cadre d'un apprentissage scolaire ou guidé. Il a donc fallu l'adapter. Cette partie explique les différentes adaptations apportées au projet d'origine et pourquoi elles sont nécessaires pour remplir les objectifs.

Dans les adaptations opérées, sont retenues une simplification de l'interface, une différenciation des rôles entre le professeur et le public cible, le masquage des scripts, une amélioration de la traduction en français et la sauvegarde sur les serveurs du projet courant.

5.2.1 L'interface

Comme expliqué précédemment, Snap! n'a pas une vocation didactique de groupe et a un public cible plus large. Il a des menus pour des fonctionnalités de gestion de l'ordonnanceur, des options d'affichage, des paramètres d'édition, etc. Certaines de ces fonctionnalités sont trop complexes et/ou peuvent distraire inutilement le public visé par Rsnap.

Un nettoyage en profondeur de l'interface de Snap! a été opéré afin de laisser uniquement les menus utiles. Toutefois, ils n'ont pas été effacés, mais masqués afin qu'ils puissent être utilisés par les professeurs, comme cela est analysé dans les rôles (section 5.2.2).

Une autre adaptation de l'interface est l'ajout d'un menu pour les interactions avec la plateforme web. Ce menu contient des boutons tels que :

- "sauvegarder sur le serveur", pour enregistrer le travail de l'élève sur la plateforme ;
- "description", pour permettre de retrouver le résumé introductif de la mission ;
- "retour à la liste des missions", pour sauvegarder le travail puis renvoyer vers la page listant les missions sur le site web.

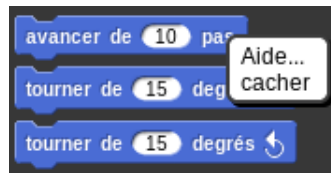


FIGURE 5.4 – Option pour cacher la définition d'un bloc

5.2.2 Les rôles

Afin de fournir une interface épurée pour les élèves, comme expliqué dans la section 5.2.1, il est nécessaire d'enlever des parties non pertinentes de l'interface de Snap!. Toutefois, ces options inutiles aux élèves peuvent l'être pour les professeurs et également pour une mission en "mode ouvert". Il est donc utile d'avoir une interface modulaire suivant son utilisation.

Pour différencier les utilisations de l'interface, la notion de rôles a été ajoutée. Deux rôles sont définis : élève et professeur. Ainsi, quand un élève ouvre un projet, Snap! est lancé avec le rôle élève, ce qui permet de ne pas afficher les options superflues. Quand un professeur souhaite modifier une mission ou en créer une nouvelle, Rsnap lance Snap! avec le rôle professeur ce qui donne accès aux fonctions avancées de Snap!.

Ces rôles permettent également la gestion du masquage de scripts. Il est intéressant qu'un professeur puisse masquer des blocs, sans que les élèves ne puissent les réafficher.

5.2.3 Masquer les scripts

Pour la création des missions, il a été nécessaire de cacher des blocs aux élèves. Par exemple, la partie de vérification du code de l'élève ou encore le code de l'environnement de la mission ne doivent pas leur apparaître.

Une possibilité de cacher des blocs existait déjà dans la zone des blocs sources (voir la figure 5.4) de l'interface. Cette fonctionnalité est étendue pour pouvoir également cacher des blocs ou des scripts dans la partie d'édition (voir la figure 2.2).

Il a donc fallu ajouter cette information dans la sauvegarde du projet. Ceci a été réalisé grâce à la balise `hidden` dans le XML (section 5.2.5).

Cette fonctionnalité étant intéressante pour le projet original, elle a été proposée dans une `pull request`. Les responsables de Snap! ont marqué leur intérêt pour cette fonctionnalité qui est toujours en cours d'étude.

5.2.4 Traduction

Ce travail se différencie des autres initiatives similaires introduites dans la section 3 par son caractère francophone. L'application était déjà traduite partiellement en français, mais une large majorité des traductions étaient incomplètes, inexistantes ou erronées. Il a donc fallu en améliorer la traduction. La traduction de l'application est différente pour la partie interface et la partie fournissant les aides.

Des aides étaient déjà disponibles en anglais. La possibilité de les avoir dans d'autres langues a été développée pour ce travail. De plus, des aides en français étaient déjà existantes dans le projet SCRATCH [42]. C'est donc elles qui ont été utilisées pour Snap!.

L'ensemble de ces aménagements a été proposé au projet original. Il a été accepté et il sera intégré après quelques modifications du serveur utilisé par Snap!.

5.2.5 Sauvegarde sur le serveur

Comme il est nécessaire d'interfacer l'environnement de programmation avec un site web, l'implémentation de certaines fonctionnalités d'import-export spécifiques à Rsnap a été réalisée.

Il faut disposer d'un fichier XML pour sauvegarder les missions sur le serveur. Les fonctions existantes ont été adaptées pour permettre d'avoir ce fichier et de pouvoir l'envoyer au serveur.

5.3 Rsnap

Cette section traite de l'implémentation de la solution Rsnap ainsi que d'une présentation de son utilisation.

5.3.1 Implémentation

Comme expliqué en section 2.3, Ruby on Rails (Rails) possède une architecture Model-Vue-Contrôleur. Cette section va présenter l'application en se basant sur ce modèle d'architecture. La première partie explique le modèle, ensuite les contrôleurs sont abordés. Enfin, les vues sont présentées.

Modèles

Les concepts utiles pour développer l'application sont ceux présentés par les différents modèles dans la figure 5.5.

User Ce modèle contient les informations nécessaires pour identifier les utilisateurs (`firstname`, etc) et pour les authentifier (`encrypted_password`,

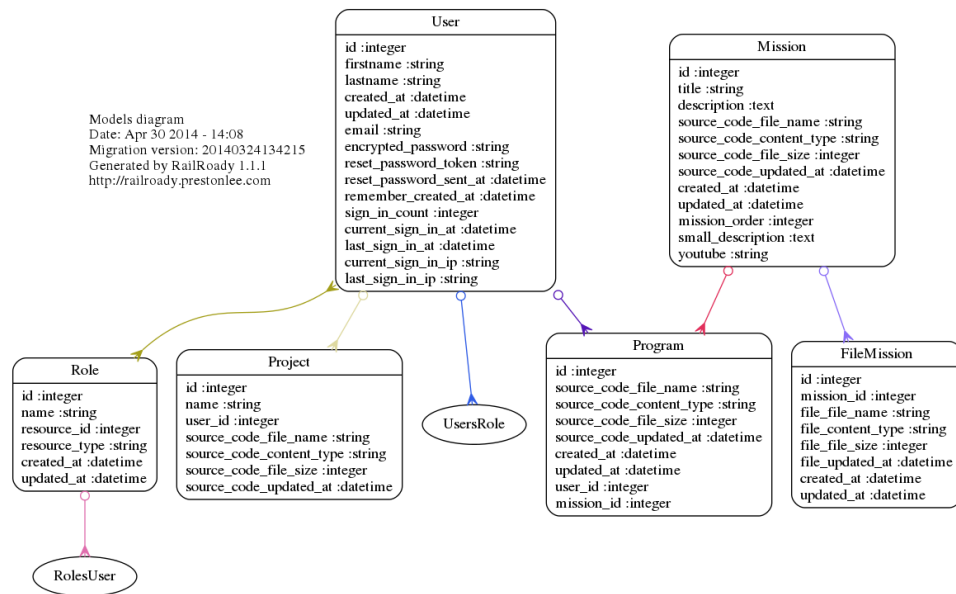


FIGURE 5.5 – Modèles de Rsnap

`current_sign_in_ip`, etc). Les utilisateurs peuvent posséder plusieurs rôles, programmes et projets.

Role Les rôles permettent de donner des attributs aux utilisateurs. Ce modèle a été généré automatiquement par Rolify (section 2.3.3). Dans le cas de Rsnap, deux types d'utilisateurs ont été créés : **admin** et **teacher**. Ces deux rôles sont globaux et ne sont donc pas rattachés à une ressource spécifique.

Les rôles sont utiles en conjonction avec les autorités (section 2.3.3) pour donner des droits supplémentaires à ces types d'utilisateurs.

Mission Les missions représentent tout ce qui est nécessaire pour avoir un exercice. Une mission comporte donc un titre, une description avec des images et une vidéo pour expliquer à l'élève ce qu'il devra faire. De plus, elle comporte le code source initial de l'exercice. Généralement, celui-ci contient le squelette initial du programme pour l'élève et les tests pour vérifier le programme.

FileMission Les fichiers liés à une mission sont les différentes images qui composent la description.

Program Les programmes contiennent la solution d'une mission (uniquement le code source).

```

1 class Program < ActiveRecord::Base
2   include Authority::Abilities
3   self.authorizer_name = 'ProgramAuthorizer'
4
5   belongs_to :mission
6   belongs_to :user
7
8   delegate :title, :to=>:mission, :prefix=>true
9   delegate :name, :to=>:user, :prefix=>true
10
11   has_attached_file :source_code
12
13   validates_attachment :source_code, :presence=>true, :
     content_type=>{:content_type=>/text/}
14   validates :user_id, :mission_id, :presence=>true
15   validates_uniqueness_of :mission_id, :scope=>:user_id
16
17   scope :for_mission, ->(mission){where(:mission_id=>mission)}
18   scope :for_user, ->(user){where(:user_id=>user)}
19   scope :order_by_missions, ->{includes(:mission).order("
     missions.mission_order ASC")}
20
21   def self.for_mission_for_user(mission, user)
22     Program.for_mission(mission).for_user(user).first
23   end
24 end

```

Code source 5.1 – Modèle Program

Project Les projets sont identiques aux programmes, mais ne sont pas liés à une mission. Ils contiennent donc le nom du projet en plus du code source.

Exemple Program est un bon exemple de modèle Rails (code source 5.1). Seules les fonctionnalités que Active Record (section 2.3.2) ne sait pas déduire de lui-même sont présentes. Le modèle contient donc uniquement :

- le nom des modèles avec qui il est associé et la cardinalité de la relation (`belongs_to` ligne 5-6);
- la validation de certains attributs pour qu'ils respectent des contraintes (`validates` ligne 13-15);
- des méthodes de classes pour simplifier les requêtes au modèle (`scope`, `self.*` ligne 17-23).

Comme c'est visible pour les `scope`'s à la ligne 17-19, Active Record permet de faire des requêtes SQL directement en Ruby.

Contrôleurs

Les contrôleurs donnent accès aux différentes ressources. La figure 5.6 montre la liste des contrôleurs implémentés pour cette application. La ma-

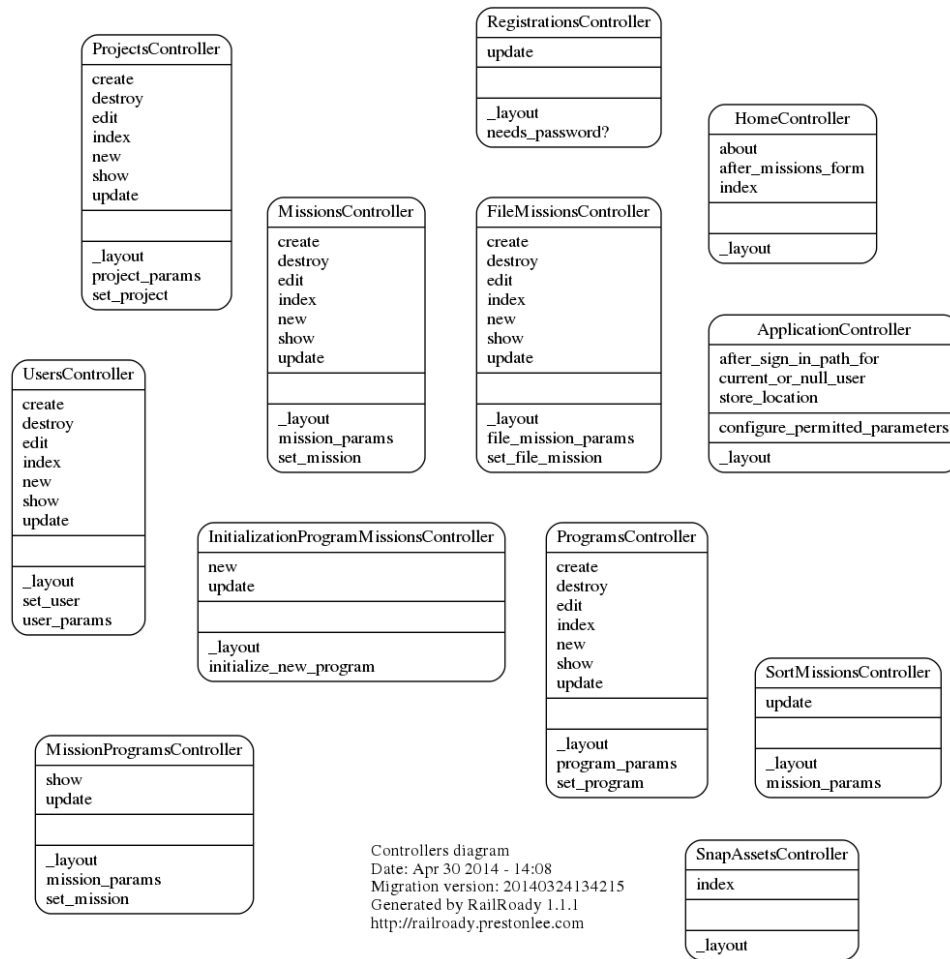


FIGURE 5.6 – Contrôleurs de Rsnap

porité de ceux-ci se rapporte directement à un modèle spécifique. Il existe néanmoins certaines ressources différentes telles que :

- les pages statiques (**HomeController**) ;
- l'ordonnancement des missions (**SortMissionsController**) ;
- la création d'un programme depuis une mission (**InitializationMissionController**) ;
- les ressources utiles à l'affichage de Snap! (**SnapAssetsController**).

Comme il est expliqué dans la section 2.3.2, les méthodes accessibles sont définies dans **routes.rb**. Les contrôleurs ont tous des ressources representational state transfer (REST) excepté **HomeController**. Ceci est visible dans **routes.rb** ou en regardant les fonctions publiques disponibles dans les contrôleurs sur la figure 5.6.

Exemple L'extrait du contrôleur `MissionsController` 5.2 montre l'usage classique d'un contrôleur.

```
1 class MissionsController < ApplicationController
2   authorize_actions_for Mission
3   before_action :set_mission, only: [:show, :edit, :update, :
4     destroy]
5   before_filter :authenticate_user!, :except=>[:index, :show]
6
7   def show
8     if params[:modal]
9       render :modal_show, :layout=>false
10    else
11      @title = "Mission : #{@mission.title}"
12    end
13  end
14
15  def new
16    @title = "Créer une mission"
17    @mission = Mission.new
18  end
19
20  def create
21    @mission = Mission.new(mission_params)
22
23    if @mission.save
24      redirect_to @mission, notice: "La mission a bien été créé
25        e."
26    else
27      @title = "Créer une mission"
28      render :new
29    end
30  end
31
32  private
33  def set_mission
34    @mission = Mission.find(params[:id])
35  end
36
37  def mission_params
38    params.require(:mission).permit(:title, :description, :
39      small_description, :source_code, :youtube)
40  end
41 end
```

Code source 5.2 – Extrait du contrôleur `MissionsController`

Grâce au méta-programmation, `authorize_actions_for Mission` permet de rajouter, sur toutes les méthodes REST, des vérifications de ce que peut réaliser ou pas l'utilisateur courant. Ces vérifications sont injectées dans les autorités (section 2.3.3). De plus, la méthode `before_filter` donne la possi-

bilité de réaliser une action supplémentaire avant chaque appel de fonction. Dans cet exemple, il faut que l'utilisateur soit authentifié pour que l'application puisse ensuite vérifier ses droits. Ceci à l'exception des actions `show` et `index` qui peuvent être publiques.

Dans une méthode publique d'un contrôleur plusieurs actions sont à implémenter :

1. traiter, si nécessaire, les paramètres fournis. Ceux-ci se trouvent dans `params` ;
2. Assigner les variables d'instance dont la vue a besoin pour s'afficher correctement ;
3. Exécuter le rendu de la page. Cette étape est facultative si la vue a le même nom que la méthode.

Vue

Les vues sont écrites en haml (section 2.3.2).

L'exemple de code 5.3 avec 5.4 montre comment la vue servant à lister tous les utilisateurs est représentée (figure 5.7).

Le code source 5.3 montre comment sont utilisées les variables d'instance créées par le contrôleur. Par exemple, la variable `@title` sert de titre 1.

De plus, Rails permet de faire le rendu d'un autre fichier de manière élégante. La ligne 14 du code source 5.3 exécute le rendu sur tous les éléments de la collection `users` grâce au fichier 5.4

Les connaisseurs de Bootstrap 2.3.2 auront reconnu l'usage intensif des classes de style qu'il propose. L'adaptativité de cet outil est visible sur la capture d'écran 5.7 : le menu supérieur a été réduit, car la taille de l'écran était trop petite pour accueillir le menu en entier.

5.3.2 Utilisation

Pour utiliser la plateforme, il faut d'abord s'enregistrer. Ensuite, suivant le profil, différentes actions sont possibles. L'utilisateur normal peut uniquement résoudre les missions dans l'ordre proposé par les professeurs. Les professeurs peuvent aussi créer des missions. Cette section présente l'utilisation de la plateforme.

Professeur Si un professeur veut lui-même réaliser un cours de programmation, il doit pouvoir faire les actions suivantes.

Créer des missions Pour créer une nouvelle mission (figure 5.8), le professeur se rend sur la page qui liste toutes les missions. Ensuite, il clique sur le bouton en bas de page "Nouvelle mission". Sur la page suivante, il donne les informations sur la mission : titre, description, explication de la


```

1 %h1
2   =@title
3
4 %table.table.table-striped.table-bordered.table-hover
5   %thead
6     %tr
7       %th Prénom
8       %th Nom
9       %th Email
10      %th
11      %th
12      %th
13    %tbody
14      = render @users
15
16 %p
17   = link_to("Créer un utilisateur", new_user_path, :class=>"btn
      btn-success")

```

Code source 5.3 – Vue index des utilisateurs

```

1 %tr
2   %td= user.firstname
3   %td= user.lastname
4   %td= user.email
5   %td= link_to "Voir", user, :class=>"btn btn-primary"
6   %td
7     = link_to "Editer", edit_user_path(user),
8       :class=>"btn btn-warning"
9   %td
10    = link_to "Supprimer", user, :method => :delete,
11      :class=>"btn btn-danger" ,
12      :data => { :confirm => "Etes-vous certain ?" }

```

Code source 5.4 – Vue d'une ligne `_user` représentant un utilisateur

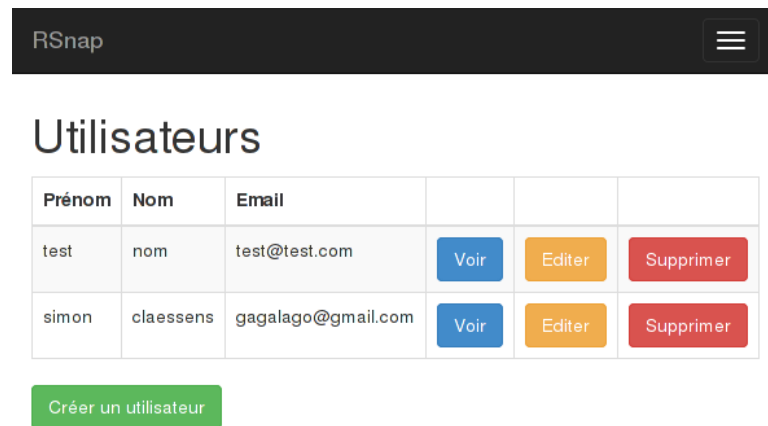


FIGURE 5.7 – Affichage de la vue présentée par le code source 5.3 et 5.4

mission, vidéo d'explication ... Il lui reste à créer la mission proprement dite. Lorsque le professeur clique sur le bouton "Créer une mission", il est redirigé vers l'interface de Snap!. Il peut alors créer tout ce qui est nécessaire à la réalisation et au test de la mission. Avant de sauver, il ne doit pas oublier de cacher les blocs et les scripts qui ne doivent pas être visibles par les élèves.

Les missions sont directement disponible à tous les professeurs pour viser à terme de créer un communauté de professeurs autour de la création de contenus adaptés aux élèves.

Ordonner les missions Toutes les missions, fournies par l'ensemble des professeurs, peuvent servir à créer un cours. Pour ordonner les missions (figure 5.9) dans un ordre pédagogique, il suffit de les glisser dans la liste.

Corriger les soumissions des élèves Pour corriger les soumissions de ses élèves (figure 5.10), le professeur se rend sur la page des programmes. Il y sélectionne le programme qu'il désire corriger et qui s'ouvre dans l'interface Snap!. Le professeur peut alors rajouter des commentaires.

Étudiant Un élève doit pouvoir réaliser les opérations suivantes.

Réaliser une mission L'élève réalise une mission (figure 5.11) en cliquant sur le nom de la mission.

Deux possibilités s'offrent à lui. Soit le bouton est bleu, ce qui signifie que la mission a déjà été commencée. L'élève la reprend là où elle s'était arrêtée. Soit, le bouton est vert et dans ce cas, une nouvelle mission commence.

Dans les deux cas, l'élève se retrouve sur l'interface de Snap!.

Réaliser un projet libre Si l'élève a envie de créer quelque chose (figure 5.12), il va dans le menu "projet libre". Il utilise Snap! à son plein potentiel et laisse libre cours à son imagination pour créer le programme de ses rêves.

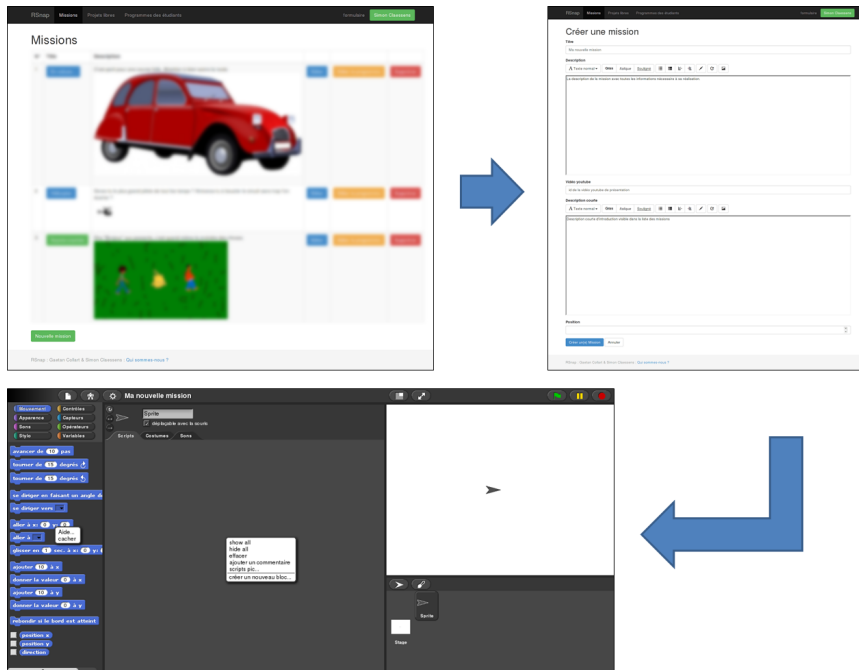


FIGURE 5.8 – Création d'une mission

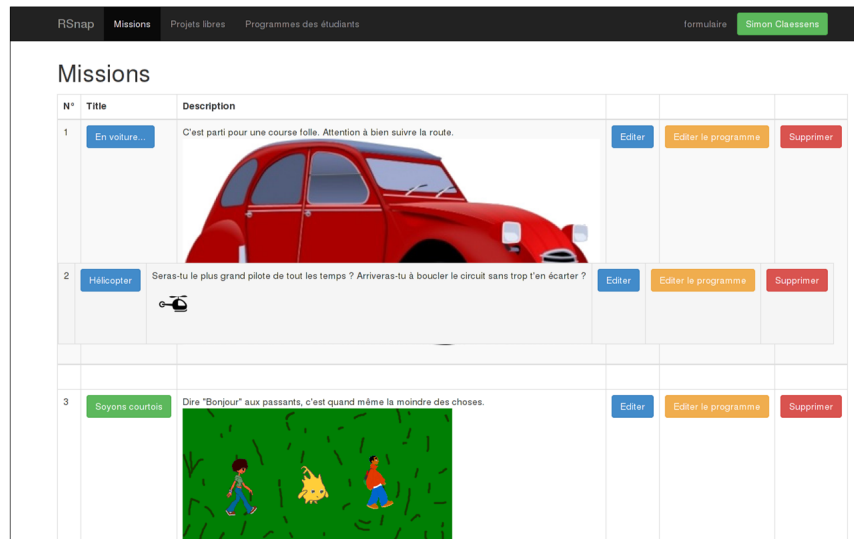


FIGURE 5.9 – Ordonner les missions

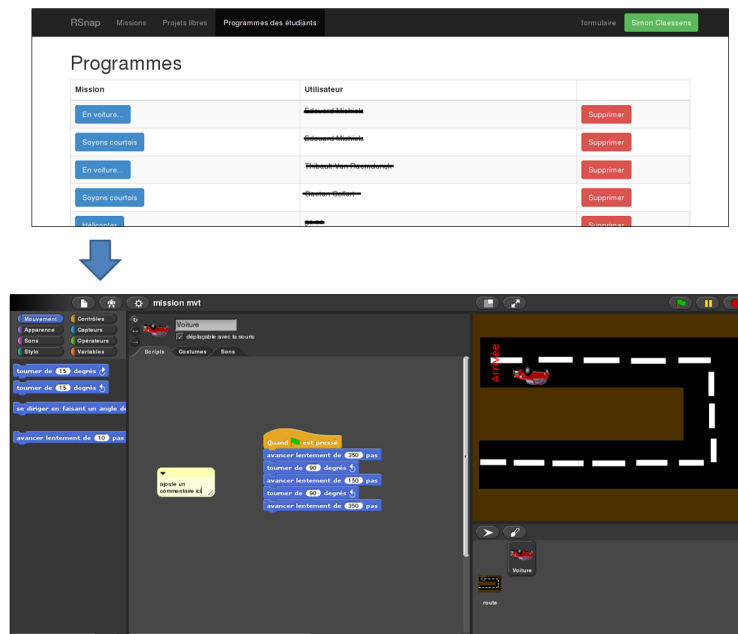


FIGURE 5.10 – Correction d'un programme

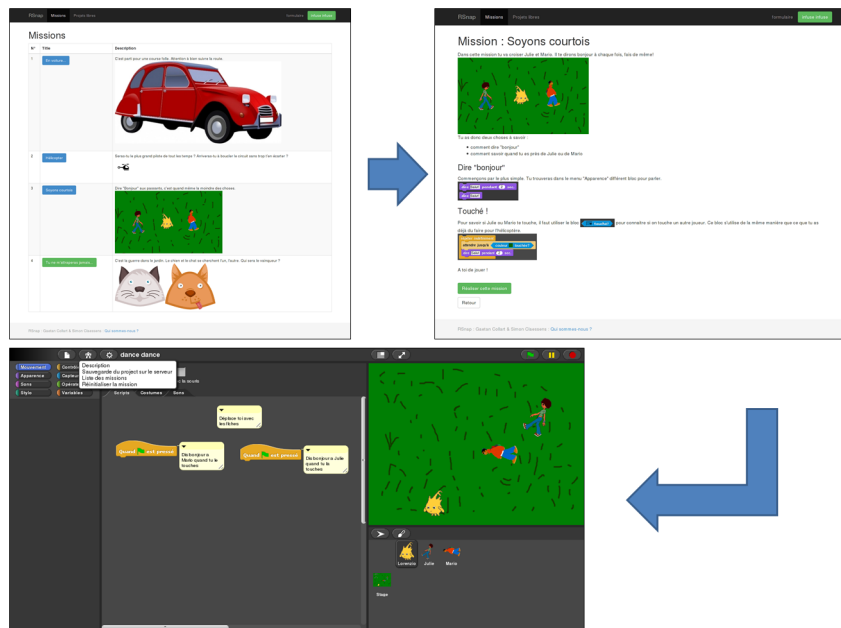


FIGURE 5.11 – Réalisation d'une mission

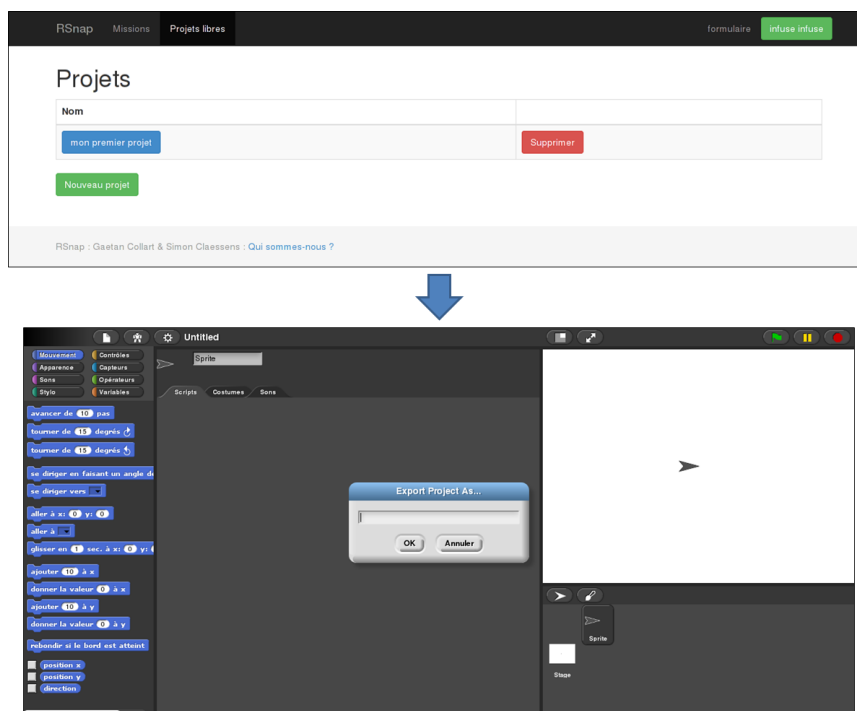


FIGURE 5.12 – Réalisation d'un projet libre

CHAPITRE 6

Validation

Ce chapitre présente les techniques de validation de la plateforme Rsnap. La validation du code est abordée dans un premier temps. Ensuite, les expériences sur des populations d'élèves et leurs analyses sont développées.

6.1 Implémentation

Dans cette section, les outils de validation du code de Rsnap sont présentés. Deux outils principaux ont été utilisés pour avoir un logiciel le mieux écrit possible (section 2.3.4) : les tests unitaires et les métriques.

6.1.1 Tests unitaires

Les tests unitaires servent à vérifier que le code fonctionne correctement. L'implémentation des tests s'est déroulée en parallèle à la création de l'application. Deux principaux types de tests ont été réalisés, ceux sur les contrôleurs et ceux au niveau de l'utilisateur. Avant de les aborder, une courte section présente les programmes utilisés pour les tests.

Outils pour les tests

Un des programmes utilisés pour les tests est Travis-CI (section 2.3.4). Ce programme est relié à Github pour que tous les tests soient exécutés à chaque commit sur le dépôt. De plus, lors d'une `pull request`, les tests sont également effectués pour savoir si l'acceptation de la branche introduira des bogues.

Une des métriques générée par les tests est la couverture du code. Cette information est fournie par Coveralls[30] qui est liée à Travis-CI pour l'acquisition des données. Cette métrique est utile pour connaître la fraction du code couvert par les tests. Elle ne donne par contre pas d'information sur le type de code testé. Il faut donc être attentif à ce que les tests couvrent effectivement les parties critiques.

Enfin, lors des tests, il est nécessaire d'avoir des données pour peupler les modèles. Ceci s'effectue grâce au gem FactoryGirls [44] qui propose un langage dédié pour écrire rapidement des fabriques. Ces dernières vont générer ces données de test pour peupler les modèles.

```
1 require 'test_helper'
2
3 class InitializationProgramMissionsControllerTest <
4   ActionController::TestCase
5
6   setup do
7     @mission = FactoryGirl.create(:mission)
8   end
9
10  def user_sign_in
11    sign_in FactoryGirl.create(:user)
12  end
13
14  test "user should get new" do
15    user_sign_in
16    get :new, mission_id: @mission.id
17    assert_response :redirect
18  end
19
20  test "nouser shouldn't get new" do
21    get :new, mission_id: @mission.id
22    assert_redirected_to controller: "devise/sessions", action:
23      "new"
24  end
25 end
```

Code source 6.1 – Test de l'accès au programme associé à une mission

Tests au niveau des contrôleurs

Un premier type de tests est réalisé grâce à RSpec [39]. Ces tests ont pour vocation de vérifier l'accès aux différentes routes (section 2.3.2) : les droits des utilisateurs et le type de retour des contrôleurs.

Un exemple de test est visible sur dans le code source 6.1. La partie `setup` utilise la fabrique spécifique pour créer une mission à chaque test. Deux tests sont ensuite exécutés et vérifient que seul un utilisateur authentifié peut réaliser une mission.

Tests au niveau de l'utilisateur

Un deuxième type de tests est réalisé grâce à Cucumber (section 2.3.4). Ces tests s'effectuent à partir du point de vue de l'utilisateur. Ils assurent le bon fonctionnement des fonctionnalités demandées par l'utilisateur ainsi que des ressources qui en dépendent. Ces tests sont donc de haut niveau, car ce n'est que via les interactions faites sur l'interface que des erreurs internes à l'application peuvent être trouvées.

Gerkin est utilisé pour écrire les scénarii en langage naturel. Le code source 6.2 montre différents scénarii décrivant l'accès aux missions. Le haut


```
1 Feature: Mission Access
2   Student can access missions only in a certain order.
3
4   @ok
5   Scenario: Access of description of a mission
6     Given a user is authenticated
7     And he is on the mission page
8     When he visit a mission
9     Then he can see the description of these mission
10
11   @ok
12   Scenario: Access on snap to solve a mission
13     Given a user is authenticated
14     And he is on some mission description page
15     When he visit the realisation of these mission
16     Then he can see the snap
17     And he can see the begining of these mission
18
19   @wip @current
20   Scenario: Save a program
21     Given a user edit a program
22     When he save the modified program
23     Then he can reload the program without loosing anything
24
25   @wip
26   Scenario: View only the already succeeded and one next
27     missions
28     Given a user is authenticated
29     And he have already complet some missions
30     When he is on the mission page
31     Then he should see already succeeded missions
32     And he should see the next mission to try
33     But he can't see other missions
34
35   @wip
36   Scenario: View only the first mission for new user
37     Given a new user is authenticated
38     When he is on the mission page
39     Then he should see only the first mission
```

Code source 6.2 – Tests de l'accès aux missions

```
1 Given(/^he is on some mission description page$/) do
2   @mission = FactoryGirl.create(:mission)
3   visit mission_path(@mission)
4 end
5
6 When(/^he visit a mission$/) do
7   click_on("show-mission-#{@mission.id}")
8 end
9
10 Then(/^he can see the description of these mission$/) do
11   page.find("h1").has_text?(@mission.title)
12   current_path.should == mission_path(@mission)
13   page.should have_text(@mission.title)
14   page.should have_text(@mission.description)
15 end
```

Code source 6.3 – Extrait de l'implémentation des tests de l'accès aux missions

niveau de Gherkin est visible dans cet exemple. En effet, toute personne anglophone peut comprendre ce que testent les différents scénarii.

L'extrait de code 6.3, montre comment sont implémentés les scénarii précédents. La façon dont est piloté un navigateur pour réaliser les tests est aussi visible via les méthodes : `click_on`, `current_path`, `visit...`

6.1.2 Utilisation de métriques

Un autre style d'outil donne des informations sur le projet via son code source. Deux services ont principalement été utilisés.

Gemnasium

Gemnasium (section 2.3.3) permet d'avoir un service de veille qui vérifie si les dépendances de l'application sont bien à jour et qu'elles ne présentent pas de risque de sécurité.

rails_best_practice

rails_best_practice (section 2.3.1) fournit quant à lui une information plus complexe à traiter. En effet, il renseigne les endroits dans le code source qui ne respecteraient pas le "rails way" et en fournit des exemples de réécriture. Cet outil est donc principalement utile en début de projet quand les auteurs ne connaissent pas encore bien quelles sont les bonnes pratiques.

6.2 Expériences

Cette section aborde les expériences réalisées pour ce travail. Celles-ci commencent par un après-midi chez KidsCode. Vient ensuite le "Printemps des Sciences" pendant lequel environ 80 élèves ont testé l'application. Enfin, une analyse des expériences à travers des formulaires remplis par les élèves et les professeurs est détaillée.

6.2.1 KidsCode

La première expérience s'est déroulée chez KidsCode. Cette organisation a été présentée dans la section 3.2.4. C'est une petite initiative locale qui apprend la programmation à 10 enfants âgés de 10 à 14 ans.

Contexte

Il est important de définir le contexte et le public de l'expérience qui en influencent considérablement son analyse.

Lors de cette expérience, le public était composé de 10 enfants de 10 à 14 ans ayant déjà fait une demi-année de programmation dans le cadre du projet KidsCode. Le niveau de ces enfants n'est donc pas à négliger. Ils sont habitués à exécuter un programme et maîtrisent les principaux concepts de la programmation.

Buts

Le but poursuivi dans cette expérience est de valider l'utilisabilité de la plateforme et également le niveau de difficulté des missions proposées.

Un autre but était de familiariser les expérimentateurs avec des utilisateurs en petit groupe, en vue de préparer la seconde expérience. Il fallait donc aussi tenir compte que l'aspect pédagogique entre en ligne de compte dans le cadre de l'utilisation de l'application. Certains paramètres ont pu être identifiés et réfléchis pour leurs implications dans l'expérience ultérieure.

Déroulement

L'expérience s'est déroulée pendant un peu plus d'une heure. A la suite des activités habituelles de KidsCode, la dernière heure y était consacrée.

Le début de l'expérience a été marqué par une perte d'attention et la dissipation des enfants. Une fois le groupe repris en main, les jeunes ont directement montré beaucoup d'intérêt, aussi bien pour l'interface qu'ils découvraient, que pour la compétition intergroupe qui s'est rapidement mise en place.

Comme expliqué dans la partie précédente 6.2.1, le public avait déjà de bonnes notions de programmation par rapport au public visé par ce mémoire.

Les premières missions ont donc été rapidement réalisées. Les principaux points de ralentissement étaient des problèmes de lecture des consignes. En effet, les auteurs n'avaient prévu que des consignes écrites. Or, les jeunes ne prenaient pas le temps de les lire et ne savaient donc pas quoi faire. Il semble que ce soit l'impatience des jeunes à mener les missions plus que leur niveau de lecture qui détermine leur comportement.

La compétition que les jeunes ont mise en place dès le début a eu un effet bénéfique pour leur évolution, car elle leur donnait la motivation de réaliser les défis proposés. Ceci était fort visible à chaque passage de mission. Chaque fois qu'un groupe finissait sa mission, il était important pour lui de le signaler aux autres. Cela lui donnait une satisfaction qui le poussait à réaliser la mission suivante.

Dans les délais impartis, alors que tous sont arrivés à la mission finale du chien et du chat, personne n'a réussi à la finir.

Quand a sonné la fin de l'expérience, beaucoup de jeunes nous ont demandé comment faire pour montrer à leur famille ce qu'ils avaient réalisé et comment faire pour continuer la mission finale.

Analyse

Dans l'ensemble, l'expérience s'est très bien déroulée et les participants ont beaucoup apprécié.

Cependant pour tenir compte de certains paramètres, des améliorations ont été apportées à l'application. Ces modifications sont abordées dans la suite de cette partie.

Changement dans les missions La mission **répète et répépète** a été retirée, car elle s'est avérée peu accrocheuse pour les enfants et peu intéressante par rapport aux concepts introduits. Un personnage devait répéter ce que disait un autre. Pendant cette mission, beaucoup d'enfants se sont dispersés parce qu'ils s'embêtaient et que ce n'était pas assez concret. Cette mission avait pour but de faire découvrir les fonctions d'affichage de textes.

Elle a été remplacée par **Soyons courtois** décrit en section 5.1.4. Cette nouvelle mission est beaucoup plus dynamique que la précédente, car elle permet à l'élève de se déplacer et d'utiliser les capteurs de collisions introduits dans la mission précédente.

Ajout d'un bouton "réinitialiser la mission" Lors de l'expérience, un groupe d'enfants a réussi à corrompre l'environnement de la mission suite à une série obscure d'opérations. Pour palier à ce genre de risque, un bouton "réinitialiser la mission" a été rajouté dans l'interface de Snap! BYOB (Snap!). Il permet de réinitialiser la mission courante.

Ajout de vidéo d'introduction Comme observé dans cette expérience, les jeunes ont tendance à ne pas lire les textes explicatifs des missions. Il peut en résulter un manque de productivité. Une vidéo d'introduction a donc été rajoutée au début de chaque mission (figure 5.11). Cette vidéo explique le but de la mission et donne les instructions. En cas d'oubli, un résumé écrit est toujours présent.

6.2.2 Printemps des Sciences

Cette expérience s'est déroulée dans le cadre de la semaine Scienceinfuse. Des écoles du fondamental et du secondaire viennent participer à des animations dans les universités. L'expérience s'est déroulée avec quatre groupes d'enfants. Ces activités duraient une heure trente y compris les temps d'installation et de prise en charge. Les élèves étaient en programmation par paire (section 3.4.2) et donc à deux devant un ordinateur.

Contexte

Lors de cette expérience, 74 élèves âgés de 11 à 13 ans étaient répartis en 4 classes, deux de sixième primaire et deux de première humanité.

L'origine des jeunes est également variée. Il y a eu des classes de Chimay, de Bousval et d'Ottignies, ce qui semble être un échantillon suffisamment représentatif.

Buts

Le but principal de cette expérience était de confronter l'application à son usage réel dans des classes d'élèves. Il fallait pouvoir observer comment elle se déroule en réalité et en ressortir une analyse des besoins spécifiques de l'application.

Un autre but poursuivi était l'évaluation de l'âge idéal et du niveau de difficulté des missions pour des néophytes. En effet, lors de l'expérience précédente, le public était volontaire et avait déjà des connaissances en programmation. De par ce fait, le niveau des missions devait être affiné par rapport au public visé.

Les auteurs désiraient aussi observer les réactions et l'intérêt des élèves pour la solution proposée.

Un dernier but était évidemment de récolter l'avis des élèves et des professeurs.

Déroulement

Le déroulement de cette expérience se divise en deux parties. La première présente le déroulement de l'activité commune aux écoles participantes. En-

suite vient une explication particulière à chaque école pour en détailler leurs spécificités.

Déroulement général des activités Lors de la mise en place de l'activité, une petite démonstration de l'interface a été présentée pour familiariser les élèves à Snap!.

Suite à cela, ils ont été invités à créer un compte par groupe de deux (par ordinateur).

Une fois la première vidéo passée, les élèves ont entamé la première mission.

La première mission était celle de la voiture, décrite à la section 5.1.2. Les élèves devaient y aborder un concept difficile, à savoir la structuration mentale de leur script. En effet, la voiture revenait à son point de départ à chaque lancement du script. Finalement, après quelques essais et erreurs, ils ont tous réussi cette mission.

Une fois la première mission finie, les auteurs ont dû rappeler aux élèves comment la sauver et comment revenir à la liste des missions. Ensuite, les élèves retrouvaient seuls comment continuer.

Une question récurrente des élèves était de savoir si leur mission avait bien été enregistrée sur le serveur.

À la fin de l'expérience, tous les participants ont reçu un diplôme avec l'adresse web de l'application, afin qu'ils sachent continuer leur programme chez eux.

École Sainte-Marie La première classe à avoir testé l'application est une sixième primaire de l'école Sainte-Marie de Bousval. Le groupe était composé d'une petite vingtaine d'élèves.

L'activité s'est déroulée avec une petite contrariété informatique. En effet, l'interpréteur JavaScript des machines mises à disposition était très lent. Ceci n'a pas compromis le déroulement de l'activité, mais fut, à quelques moments, une source de frustration pour certains élèves très enthousiastes.

À la fin du temps imparti, tous les élèves avaient atteint la dernière mission décrite en section 5.1.5. Ils n'ont généralement pas fini l'étape de déplacement du personnage, mais étaient bien avancés en ce sens.

Athénée Royal Paul Delvaux Cette école est venue avec une classe de sixième primaire également. Pour cette expérience, le problème de réactivité de l'interface a été corrigé par un changement de navigateur.

Par rapport à l'école de Bousval, les élèves semblaient avoir moins de facilité à comprendre et être moins autonomes. Cependant, ils sont arrivés à

peu près aux mêmes résultats que ceux de Bousval.

L'enthousiasme des jeunes était également au rendez-vous et ils se sont bien amusés.

Collège Saint Joseph Les deux dernières classes à participer à l'expérience étaient du collège Saint Joseph de Chimay accompagnés par leurs professeurs de mathématique.

Comme ces deux classes avaient un niveau de première humanité, les enfants étaient plus autonomes. L'expérience s'est déroulée comme pour les primaires. La première mission était la plus laborieuse, mais une fois celle-ci passée, le reste a été plus aisé. Ces enfants étant plus grands, il a semblé aux auteurs qu'ils présentaient une capacité d'apprentissage plus élevée. En effet, ils ont été beaucoup plus rapides pour réaliser les missions. Ceci leur a permis d'avancer davantage dans la dernière mission. Certains groupes ont même réalisé un programme fonctionnel pour cette dernière mission : il ne manquait que le score à implémenter.

Dans ces classes une différence de niveau au sein des groupes a été observée. En effet certains duos étaient très autonomes alors que d'autres se rapprochaient plus des groupes de l'enseignement fondamental.

Analyse

Compte tenu des remarques précédentes, l'ensemble de ces expériences s'est globalement bien déroulé. Les objectifs ont été approchés. Le niveau des missions correspondait à ce qui était souhaité. La plupart des enfants ont très vite et bien pris en main l'interface de Snap!. Les modifications apportées par la première expérience ont montré toutes leurs utilités. En outre, le bouton "réinitialisation" s'est avéré facile à utiliser. La mission **Soyons courtois**, voir section 5.1.4, a été appréciée des enfants et a rempli pleinement son rôle.

Certains points sont un peu plus détaillés tels que les vidéos d'introduction, les tranches d'âge et les améliorations qui ont été trouvées grâce à cette expérience.

Vidéo Une crainte était qu'avec le lecteur YouTube, ils regardent d'autres vidéos après avoir regardé leur vidéo d'introduction à la mission. Il n'en fut rien et la curiosité des élèves était assez forte pour les garder dans l'application.

Tranche d'âge Un des objectifs de ce travail est que les enfants utilisent cette application de manière quasi-autonome. Dans ce sens, durant les expériences, les auteurs ont constaté que les enfants de début d'humanité sont plus aptes à travailler de cette manière que les élèves de primaire : moins d'appels à l'aide, plus de persévérance, meilleure prise en charge individuelle,

etc. En conséquence, l'application pour des élèves du fondamental, nécessite soit une plus grande maîtrise de la matière soit plus d'encadrants.

Amélioration Lors de cette expérience, les auteurs ont été particulièrement attentifs au retour des enfants et à leurs réactions spontanées. Il ressort de ces observations quelques améliorations telles que :

- ajouter un message d'information sur la sauvegarde des projets ;
- diminuer le taux de rafraîchissement de la fenêtre ;
- diviser la mission hélicoptère en deux missions séparées : la boucle "répéter indéfiniment" et la condition "si".

Reconnexion Lors de la semaine de l'expérience de Sciences infuse, il y a eu 51 reconnexion indépendante des animations. Ceci montre que les enfants sont retourner sur le site, très probablement pour montrer ce qu'il avait fait à leur parents. Le temps moyen de ces sessions est d'environ 8 minutes.

Cependant peu de reconnexion ont été observées lors des semaines suivantes. Ceci peut s'expliquer par le fait que le contenu actuel du site requière une supervision pour être abordable par des enfants qui n'ont pas de connaissance préalable en programmation.

Impression générale Par des expériences ludiques menées en une heure, les enfants ont appris des concepts basiques de la programmation. Les auteurs se sont rendu compte que beaucoup d'enfants n'avaient pas vu le temps passer.

6.3 Formulaire

Un formulaire a été complété par chaque enfant et chaque professeur à la fin de l'activité. Dans cette partie, les formulaires sont analysés. Les données complètes, les graphiques et les commentaires des enfants sont disponibles en annexe A.2.

6.3.1 Création du formulaire

Le formulaire a deux objectifs principaux : connaître nos participants et connaître leur ressenti par rapport aux missions.

Participants La première partie du formulaire (figure 6.1) est utile pour apprendre des informations générales sur les participants telles que leur âge, leur niveau de connaissance en informatique et en programmation avant et après l'expérience.

Compte rendu des missions

Quand tu as fini les différentes missions, merci de nous donner tes impressions via ce formulaire.

Informations Générales

Différentes questions pour mieux te connaître.

Je suis à ...
Indique nous ton école.

Je suis en ...

Je savais utiliser un ordinateur avant de venir ?
J'ai facile à utiliser l'ordinateur (souris, clavier...) ?

1 2 3 4

Plutôt difficilement ☐ ☐ ☐ ☐ Plutôt facilement

Je sais mieux utiliser un ordinateur maintenant ?

1 2 3 4

Non, pas plus qu'avant ☐ ☐ ☐ ☐ Oui, beaucoup mieux

Je savais ce qu'était la programmation avant de venir ?

1 2 3 4

Non, je n'en avais jamais entendu parler avant ☐ ☐ ☐ ☐ Oui et j'avais même déjà programmé

Je comprend mieux ce qu'est la programmation maintenant ?

1 2 3 4

Non, pas plus qu'avant ☐ ☐ ☐ ☐ Oui, beaucoup mieux

Terminé à 12 %

Fourni par Google Forms

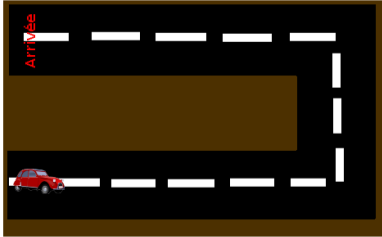
Ce contenu n'est ni rédigé, ni cautionné par Google.

[Signaler un cas d'utilisation abusive](#) - [Conditions d'utilisation](#) - [Clauses additionnelles](#)

(a) Formulaire pour connaître les participants

Compte rendu des missions

Mission : En voiture...



J'ai aimé réaliser cette mission ?

1 2 3 4

Je me suis embêté ☐ ☐ ☐ ☐ Je me suis bien amusé

Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?

1 2 3 4

Difficilement ☐ ☐ ☐ ☐ Facilement

Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?

1 2 3 4

Difficilement ☐ ☐ ☐ ☐ Facilement

Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?

1 2 3 4

Difficilement ☐ ☐ ☐ ☐ Facilement

J'ai des remarques à transmettre.
Explique nous ce que tu as aimé, détesté, ce qui était bien fait, ce qu'on devrait encore améliorer...

Terminé à 37 %

Fourni par Google Drive

Ce contenu n'est ni rédigé, ni cautionné par Google.

[Signaler un cas d'utilisation abusive](#) - [Conditions d'utilisation](#) - [Clauses additionnelles](#)

(b) Formulaire pour connaître le ressenti des participants à propos de la première mission

FIGURE 6.1 – Formulaire fourni aux participants

Missions La deuxième partie du formulaire est un questionnaire identique pour chaque mission (figure 6.1). Celui-ci permet d’obtenir des retours sur l’appréciation de ces derniers. Ces retours portent sur les différentes parties des missions : les explications, la réalisation et leurs appréciations générales. Les formulaires sont collectés dans le but d’améliorer les missions.

6.3.2 Protocole de récolte

C’est durant les dix dernières minutes de l’activité que les élèves ont dû remplir le formulaire. Ils devaient le remplir par binôme. Il y a eu 16 formulaires remplis par des élèves du fondamental et 20 par des binômes du secondaire.

6.3.3 Analyse des formulaires

L’analyse des formulaires se réalise en suivant deux axes : le niveau des enfants en informatique et l’évaluation des missions. Les formulaires comprenaient des évaluations sur une échelle allant de 1 la moins bonne à 4 la meilleure.

Niveau des enfants en informatique

Une question du formulaire portait sur la connaissance de l’informatique et de la programmation. Les schémas montrent comment les enfants autoévaluent leurs compétences en informatique (figure 6.2) et en programmation (figure 6.3). Ces tableaux sont analysés dans la suite de cette section.

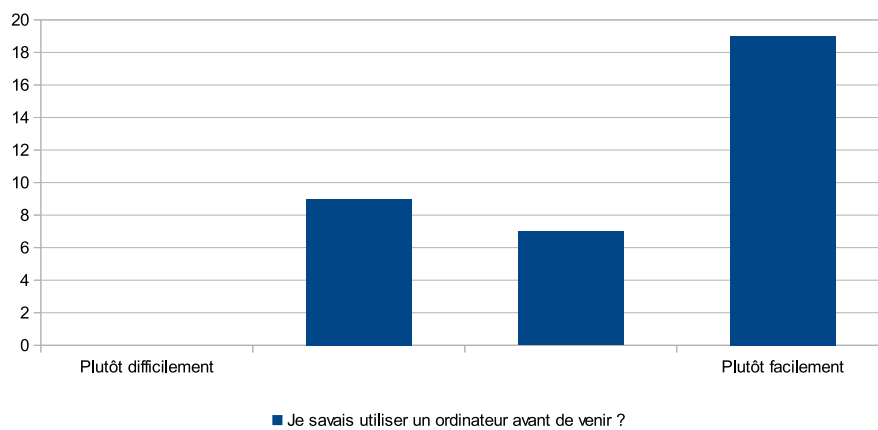
Les enfants évaluaient qu’ils avaient déjà une bonne connaissance en informatique (moyenne 3,3), les réponses étaient fort similaires (variance 0,7). Une question demandait s’ils pensaient que l’activité a amélioré leur connaissance. La moyenne est de 2,4 et n’ont donc pas l’impression de s’être améliorée, mais les réponses sont beaucoup plus dispersées (variance 1,3).

Les mêmes questions ont été posées sur leurs connaissances en programmation. L’évaluation de leurs connaissances avant l’activité était beaucoup plus faible (moyenne 1,8). Leur progression suite à l’activité est beaucoup plus marquée (3,3). Cette moyenne représente l’avis d’une forte majorité (variance 0,8).

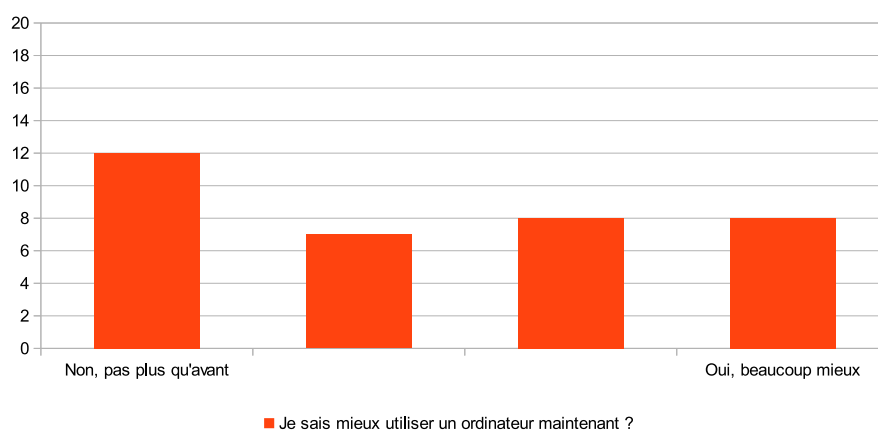
Ceci montre qu’ils estiment s’être fortement améliorés en programmation, mais moins dans la maîtrise de l’ordinateur.

Évaluation des missions

Les participants ont également évalué séparément chaque mission. Le graphique représentant cette évaluation est sur la figure 6.4. Le score des missions est présenté dans le tableau 6.2.



(a) Avant l'expérience

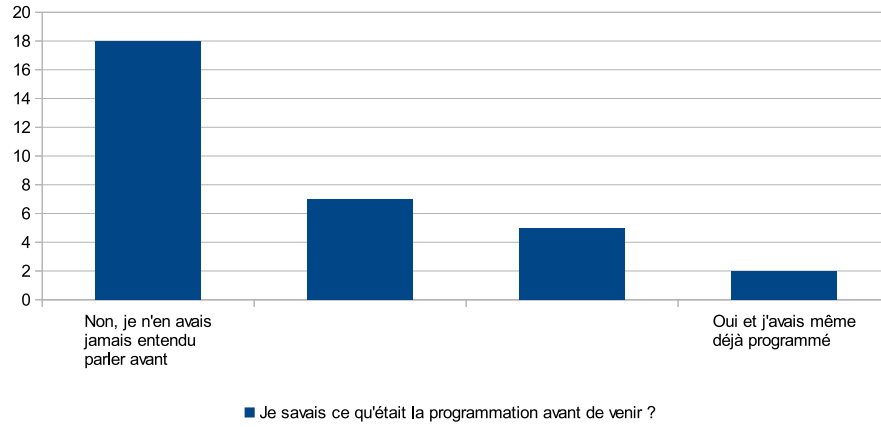


(b) Amélioration grâce à l'expérience

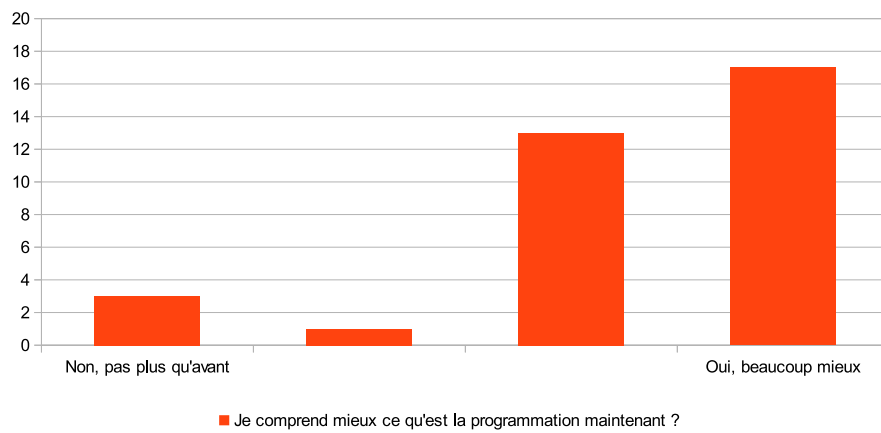
FIGURE 6.2 – Auto-estimation du niveau en informatique des participants

	Utilisation d'un ordinateur		Connaissance en programmation	
	Avant	Amélioration	Avant	Amélioration
Moyenne	3,3	2,4	1,8	3,3
Variance	0,7	1,3	1,0	0,8

TABLE 6.1 – Score des missions



(a) Avant l'expérience



(b) Amélioration grâce à l'expérience

FIGURE 6.3 – Auto-estimation de la connaissance en programmation des participants

Mission	Général		Fondamental	Secondaire
	Moyenne	Variance	Moyenne	Moyenne
En voiture	3,5	0,5	3,6	3,5
Hélicoptère	3,1	1,1	3,1	3,1
Soyons courtois	2,8	1,3	2,2	3,0
Chien et chat	2,8	1,6	3,4	2,6

TABLE 6.2 – Score des missions

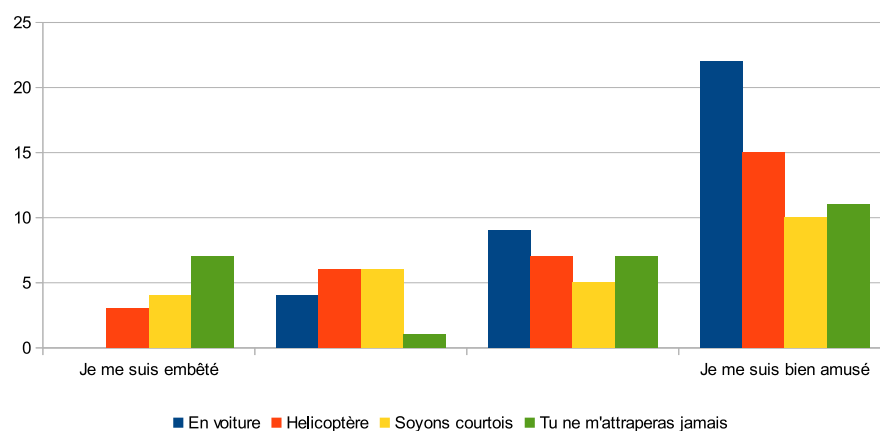


FIGURE 6.4 – Évaluation des missions par les enfants

En voiture La première mission (section 5.1.2) a été la plus laborieuse. En effet, les expérimentateurs ont dû fournir de l'aide à de nombreuses reprises pour débloquer les participants. Cependant, cette mission a le plus haut taux d'appréciation des enfants, avec une moyenne de 3,5 ceci avec une variance très faible de 0,5 ce qui montre un accord général des enfants (tableau 6.2). Est-ce dû à la facilité de compréhension de cette mission ? Est-ce le graphisme ? Ou encore son côté concret ? Les formulaires n'apportent pas assez d'éléments de réponse.

Hélicoptère La seconde mission (section 5.1.3) a été appréciée avec le deuxième score de 3,1 et une variance moyenne de 1,1 (tableau 6.2).

Comme cité plus haut (section 6.2.2), elle pourrait être séparée en deux missions. En effet, beaucoup d'élèves ont eu besoin de l'aide des auteurs pour cumuler les deux concepts abordés en une fois.

Soyons courtois La troisième mission (section 5.1.4) est la moins appréciée du fondamental avec une moyenne à 2,2 contre 3,0 pour le secondaire (tableau 6.2).

Une différence par rapport aux autres missions est que la description de celle-ci ne comporte pas de vidéo explicative. Cette différence pourrait expliquer la différence de résultats entre le fondamental et le secondaire.

Tu ne m'attraperas jamais La quatrième mission (section 5.1.5) a passionné les élèves, mais ils ont été frustrés par le manque de temps. Personne n'a réussi à la finir. Elle obtient donc une note moyenne de 2,8 mais avec une

dispersion des réponses grandes (variance 1,6). Il faudrait peut-être envisager de guider davantage les élèves dans cette mission. Par exemple, tous les blocs y étaient accessibles, ceci afin que les élèves puissent découvrir ce qu'il est possible de faire avec Snap!. L'analyse des commentaires montre que les élèves se perdent dans la masse de blocs disponibles.

Une autre possibilité serait d'augmenter le temps imparti à l'expérience tout en tenant compte de la capacité de concentration des enfants. Une durée qui semble intéressante serait deux plages de cinquante minutes pour l'entièreté des missions proposées.

Cette mission est aussi appréciée différemment par les deux tranches d'âge étudiées. En effet, le fondamental a mieux aimé réaliser cette mission que le secondaire. Cette différence peut être expliquée par la part de créativité laissée aux participants notamment dans le choix de l'image pour les personnages.

Autres questions du formulaire

Les autres questions n'apportent pas d'informations supplémentaires, elles permettent d'affiner l'analyse des missions et ont permis d'orienter les solutions proposées. L'intégralité des questions et des réponses se trouvent en annexe A.2.

Appréciation des professeurs

En ce qui concerne les retours des professeurs, voir annexe A.3, ils étaient tous, sans exception, très positive. Les professeurs d'humanité ont montré beaucoup d'intérêts pour l'utilisation de l'application dans leurs cours.

Les professeurs trouvaient que les consignes étaient bien présentées et adéquates pour les tranches d'âges visées, ceci montre que l'orientation prise en section 4.1 est valide.

6.3.4 Avis général sur les expériences

Ces expériences permettent de montrer où il est encore nécessaire de fournir un travail (chappitre 8).

Le formulaire fournit des informations assez détaillées sur le ressenti des enfants vis-à-vis de chaque mission. De manière générale les chiffres obtenus lors de ces expériences montrent que les enfants ont aimé participer et les retours écrits confirme cette tendance, voir annexe A.2.

Le nombre de reconnexion au site lors de la semaine montre que plus d'un enfant sur deux s'est reconnecté au site dans les quelques jours suivants. Ceci montre un intérêt marqué de la part des enfants et une fierté par rapport à leurs travaux.

Conclusions

Ce chapitre conclut ce mémoire en reprenant les objectifs établis au chapitre 1.4. Ces objectifs s'articulaient en trois volets majeurs : plateforme d'apprentissage, contenu et langue et il se finit par un bilan final de ce mémoire.

7.1 Plateforme d'apprentissage

L'objectif était de fournir une plateforme qui permet d'enseigner la programmation aux jeunes de 10 à 14 ans. Cette plateforme doit être disponible partout et être facile d'utilisation. Elle doit également pouvoir stocker des informations pour qu'elles soient disponibles lors de prochaines utilisations.

Cet objectif se décompose en plusieurs parties :

La portabilité La plateforme Rsnap se présente sous la forme d'un site web [2] pour pouvoir y accéder grâce à n'importe quel ordinateur, tablette, etc. Elle est développée en Ruby on Rails (Rails) et JavaScript qui sont des technologies ayant fait leurs preuves, ceci assure une meilleure pérennité à la plateforme. Cependant, pour pouvoir bénéficier de toutes les fonctionnalités utilisées par Snap! BYOB (Snap!), il est nécessaire d'avoir un navigateur récent.

Disponibilité L'application étant un site web, une connexion à internet est donc nécessaire pour pouvoir avoir accès à la plateforme. Hormis cette contrainte, la plateforme est disponible en tout temps et tout endroit.

Gestion de classe L'application fournit déjà la possibilité d'avoir des rôles différents pour les élèves et les professeurs. Ceci permet notamment aux professeurs de gérer la progression des élèves. Cette fonctionnalité est encore un petit peu primitive pour être pleinement utilisable. Elle souffre d'un problème de cloisonnement pour le rôle des professeurs, mais est déjà utilisable pour les élèves.

L'âge des utilisateurs Les expériences, voir section 6.2, ont permis de montrer que la plateforme est adaptée à la tranche d'âge visée par ce travail. En effet, les élèves n'avaient aucun mal à passer d'une mission à l'autre, sauver leurs projets, les reprendre, etc. Ceci est le fruit des adaptations réalisées grâce aux expériences.

7.2 Contenu

Le contenu comprend toutes les ressources pédagogiques de la plateforme Rsnap, c'est-à-dire les différentes missions de Rsnap, mais également les outils pour en créer de nouvelles.

Dans le contenu il est important de différencier les missions existantes des moyens mis en place pour créer ultérieurement du contenu supplémentaire. C'est avec cette subdivision que cette section va aborder le contenu de Rsnap.

7.2.1 Missions existantes

Cette partie va détailler les concepts enseignés et leur mise en pratique dans les missions déjà proposées.

Concepts Les missions déjà présentes dans Rsnap introduisent les concepts de base de la programmation tel que les boucles, les conditions et l'enchaînement d'instructions. Certains concepts sont moins faciles à maîtriser par les enfants. Dans les missions actuelles, il serait avantageux de mieux séparer certains concepts. Par exemple, la mission de l'hélicoptère, voir section 5.1.3, introduit les boucles et les conditions, il serait plus judicieux d'en faire deux missions différentes. Malgré cela tous les enfants ont réussi à réaliser ces missions durant les expériences ce qui montre qu'elles sont déjà bien abouti.

Mise en pratique La forme des missions a fortement intéressé les élèves. Des personnages concrets et des retours directs sur leurs actions se sont avérés importants. Les élèves appréciaient, par exemple, que la voiture explose si elle sortait de la route. Ces détails permettent de garder l'enfant attentif et concentré sur les missions.

7.2.2 Création de missions

Cette partie aborde les outils mis en place pour aider à la création de missions supplémentaires.

Création de contenu Pour permettre aux professeurs d'avoir du contenu adapté à leurs besoins, Rsnap permet de créer leurs propres missions. C'est dans cette optique que des modifications ont été opérées sur Snap!, voir section 5.2. La création de missions par les professeurs a été pensée pour que ces derniers n'aient pas besoin de connaissance spécifique en programmation. En effet, ils créent leurs nouvelles missions directement dans l'interface de Snap!.

Communauté La création de missions dans Rsnap se veut communautaire. En effet comme expliqué dans la section 5.3.2, toutes missions créées sont disponibles pour tous les professeurs.

7.3 Langue

Un des points importants de ce travail est qu'il doit être en français pour pouvoir être proposé au plus grand nombre possible d'enfants francophone.

Traduction L'application Rsnap a été créée et pensée en français. Il est envisageable de traduire cette plateforme. En effet comme elle respecte bien les principes de Rails, elle est facile à traduire. Les seules parties qui ne pourraient être traduites de manière semi-automatique sont les descriptions des missions. De plus, Rsnap intègre l'interface de programmation Snap! pour laquelle une traduction partielle existait déjà. Une partie de ce travail a été de compléter cette traduction. Ce travail a d'ailleurs fait l'objet d'une soumission à la communauté de Snap! qui l'a accueillie avec beaucoup d'intérêt.

Culture Il est important de souligner que la traduction d'une application dans une langue est plus complexe que la traduction de chaque mot. Une traduction, c'est aussi adapter le contenu à la culture du public cible. La création du contenu de Rsnap s'est essentiellement inspirée de travaux anglophones, il était donc important de vérifier cette vision sur un public francophone. Les expériences nous montrent que cette adaptation a été réussie pour le contenu proposé.

7.4 Bilan final

Cette section évalue les objectifs de manière plus globale, met en lumière les apports de ce travail à l'état de l'art et se finit par une vision du futur de cette application.

Objectifs Dans l'ensemble, les objectifs fixés au début de ce travail ont été atteints, bien que ce genre de travail n'ait jamais de fin. Le résultat est une application qui est déjà utilisable et qui a été appréciée lors des expériences.

Apport En plus de proposer une plateforme d'apprentissage de la programmation pour les jeunes francophones, un des apports principaux de ce travail est de montrer que tous les outils nécessaires à sa création sont déjà disponibles individuellement. En effet, la plupart de ces outils sont le fruit d'un besoin de la société de comprendre et maîtriser les technologies qui nous entourent. Ce travail prouve que ces outils peuvent être rassemblés en une plateforme cohérente. Ce pas supplémentaire va dans le même sens que le rapport européen [1] évoqué en introduction.

Future de Rsnap Comme dit précédemment, Rsnap est une plateforme aboutie bien qu'il reste encore beaucoup d'améliorations possibles qui sont abordées dans le chapitre 8. Deux futures probables s'ouvrent

pour cette application. Soit, elle est reprise par une organisation pour fournir du contenu et du support pour les utilisateurs. Soit, une communauté se crée autour de Rsnap grâce aux besoins que comble cette application.

Futures améliorations

Ce chapitre aborde les améliorations possibles pour Rsnap. Elles sont soit liées aux objectifs qui n'ont pas été rencontrés ou à l'analyse des expériences.

8.1 Tester la réussite des missions

Certains tests sont prévus pour que l'élève connaisse la réussite de la mission. Toutefois, aucun signal n'est envoyé vers les serveurs pour leur transmettre cette réussite. Comme aucun mécanisme n'est présent pour valider une mission, la mission suivante se débloque dès le premier enregistrement de la précédente.

Modifications à apporter Le simple envoi d'une requête au serveur suffirait si la mission est réussie. La difficulté réside dans la création d'un vrai bloc disponible dans l'interface des professeurs qui servirait à informer le serveur de la réussite de la mission.

8.2 Aide à la gestion des groupes

Un des buts de ce travail est son utilisation dans l'enseignement. Ainsi, un système de gestion des élèves par classe a été pensé par les auteurs. Malheureusement, le temps imparti à ce travail a manqué pour avoir une implémentation complète de gestion de groupes.

Modifications à apporter Le principe pensé pour la gestion de classe est que chaque professeur puisse voir l'avancement de ses élèves. Pour ce faire, chaque élève doit être assigné à ses propres classes et les droits des professeurs ne doivent porter que sur leurs classes.

Concrètement, il faut rajouter dans le modèle une `classe` qui représente un cours. Celle-ci serait constituée de la description de ce cours, d'un professeur et de plusieurs élèves. Il faut bien sûr aussi modifier les droits pour tenir compte de ces différentes assignations.

8.3 Améliorations et ajout de missions

Comme mis en lumière dans la partie 6.3, les expériences ont montré que certaines missions gagneraient à être retravaillées.

Hélicoptère La mission de l'hélicoptère introduit deux concepts fort abstraits : les boucles et les conditions. Elle serait beaucoup plus facilement compréhensible par les enfants si elle était découpée en deux parties, une pour chaque concept.

Soyons courtois La mission "soyons courtois" n'est pas encore assez attractive pour les enfants. Il était suggéré dans l'analyse (section 6.3.3) de rajouter la programmation des mouvements de leur personnage. En effet, actuellement, seuls les autres personnages bougent. On pourrait envisager que le jeune programmeur ce qui est nécessaire pour déplacer son personnage avec le clavier.

Tu ne m'attraperas pas Comme développé dans l'analyse (section 6.3), la mission "Tu ne m'attraperas pas" n'est pas assez dirigée. Une mission d'introduction supplémentaire serait probablement bénéfique. Un autre problème dans cette mission est le nombre de blocs laissés à disposition des élèves. Le choix avait été fait de leur laisser un accès à tous les blocs pour qu'ils voient la pleine puissance de Snap! BYOB (Snap!). Ce choix s'est révélé inapproprié.

Ajout de missions En plus des missions proposées, il est important que d'autres missions soient créées pour avoir des cours plus consistants. Ces nouvelles missions permettraient d'apporter de nouveaux concepts, mais surtout permettraient aux enfants de pratiquer les concepts déjà appris pour mieux les intégrer. Il serait intéressant d'introduire les variables et les fonctions dans des missions futures.

8.4 Blocs d'introspection des programmes

Une autre amélioration serait une série de blocs dans Snap! fournissant des informations sur le programme. Il est déjà possible de savoir jusqu'où l'élève est arrivé et donc de lui attribuer des points en fonction. Pour aller plus loin, il serait intéressant d'avoir un compteur de blocs pour les scripts. En effet, un programme fait avec 20 blocs quand la solution optimale en compte 3 ne mérite probablement pas autant de points.

De manière générale, il pourrait être envisagé de fournir de vrais blocs aux professeurs afin de leur faciliter la création de corrections et de cotations automatiques.

Cette amélioration pourrait être couplée avec le test de réussite. Cette solution enverrait dans la requête au serveur une note et des informations sur l'avancée du programme grâce à ces blocs d'introspection.

8.5 Charte graphique

Pour que ce logiciel soit apprécié, une attention particulière au ressenti des utilisateurs est nécessaire. Un point important est donc d'avoir une charte graphique unifiée et cohérente en fonction du public visé.

Bootstrap et Snap! Actuellement, d'une part toute la partie Ruby on Rails (Rails) utilise Bootstrap de manière brute, d'autre part l'interface de Snap! n'a été modifiée que de manières fonctionnelles. Il serait judicieux d'adapter un peu la feuille de style de Bootstrap et l'implémentation de Snap! pour avoir un thème graphique unifié et fournir un environnement plus adapté.

Missions Les ressources graphiques utilisées dans les missions proviennent toutes de sources différentes. Avoir un artiste qui dessinerait les différents objets nécessaires créerait une meilleure unité entre les missions et les mettrait en valeur.

8.6 Futur de l'application

Cette partie concerne le futur de l'application Rsnap tant au niveau du code que de la génération de contenu.

8.6.1 Amélioration du code

Beaucoup de travail a déjà été fait au point de vue du code. Actuellement le code est facilement réutilisable. Il respecte les conventions Rails, est en JavaScript et est testé de manière exhaustive. Mais beaucoup d'améliorations sont encore possibles et même nécessaires pour l'utilisabilité de la plateforme.

Il est peu probable qu'une communauté technique s'y intéresse en l'état car Rsnap n'est pas encore assez concret pour attirer l'attention de ces dernières.

La plateforme constitue, toutefois, une très bonne base pour continuer et être reprise par une organisation. En effet, le travail fourni est conséquent et permettrait un gain de temps considérable. Il serait donc plus probable que dans le futur ce soit une organisation qui améliore Rsnap et qui arrive, avec une plateforme plus aboutie, à constituer une communauté technique.

8.6.2 Amélioration de l'utilisabilité

Des améliorations sur l'utilisabilité sont encore à réaliser. Le temps imparti à l'implémentation de la plateforme n'a pas suffi pour produire une solution pleinement utilisable. Il est donc peu probable que dans l'état actuel, une communauté d'utilisateurs se crée autour de Rsnap.

Cependant, comme dit précédemment, si l'utilisabilité de l'application est retravaillée et améliorée, il est très probable qu'une communauté d'utilisateurs se constitue autour de Rsnap.

Glossaire

Bloc

Un bloc est une pièce de puzzle symbolisant une opération qui est utilisée pour créer un script. 1, 5–9, 20, 22–24, 26, 30, 33, 36, 39, 40, 49, 68, 73–75, 78

Fondamental

18, 21, 59, 61, 62, 64, 67, 68, *voir* primaire

Gem

Un gem [38] est un programme ou une bibliothèque Ruby distribuée grâce à RubyGems. 5, 13, 14, 32, 53

Humanité

59, 61, 68, *voir* secondaire

Mission

Une mission est un exercice mettant en scène un problème. 3, 4, 19, 20, 26, 29–32, 35–37, 39–43, 48, 49, 51, 54, 57–62, 64, 67–71, 73–75

Modèle-vue-contrôleur

Le patron modèle-vue-contrôleur [52] est un patron destiné à répondre aux besoins des applications interactives en séparant les problématiques liées aux différents composants au sein de leur architecture respective. 5, 12, 32

Primaire

L’enseignement primaire est la première partie de l’enseignement officiel obligatoire pour les enfants typiquement de 6 à 12 ans en Belgique. 59–61

Pull request

Une `pull request` est une méthode de soumission de contribution à un projet au développement ouvert. C’est souvent la meilleure façon de soumettre des contributions à un projet à l’aide d’un système de contrôle de version distribué, comme git. 40, 53

Representational state transfer

REST est un style d’architecture pour les systèmes hypermédias distribués. Il impose une communication sans état et une interface uniforme. 13, 43

Rsnap

Rsnap [2] est l'application développée dans le cadre de ce travail. Elle est visible sur <https://rsnap.herokuapp.com/>. Le nom Rsnap provient de la contraction des deux principaux outils utilisé pour construire cette plateforme à savoir : Ruby on Rails (Rails) et Snap! BYOB (Snap!). 1, 3, 4, 29–32, 35, 39–42, 53, 69–73, 75, 76, 78

Ruby on Rails

Ruby on Rails [28] est un framework web. 3–5, 9, 12–15, 32, 41, 43, 47, 69, 71, 75, 78

Rôle

Type de personne utilisant Rsnap. Actuellement, deux rôles sont définis : élève et professeur. 3, 14, 39, 40, 42, 69

Script

Un script est un ensemble de blocs créant un programme exécutable. 6–9, 22, 23, 36, 39, 40, 49, 60, 74, 77

Secondaire

L'enseignement secondaire est la seconde partie de l'enseignement officiel obligatoire pour les enfants typiquement de 12 à 18 ans en Belgique. 19, 59, 64, 67, 68

Snap! Build Your Own Blocks

Snap! [41] est un langage de programmation visuel par glisser-déposer de blocs. iv, 3–7, 9, 33, 35, 36, 39–41, 43, 49, 51, 58, 60, 61, 68–71, 74, 75, 78

Sprite

Un sprit est une entité graphique contenant un ou plusieurs scripts associés. 6, 7, 23, 24

Bibliographie

- [1] Walter Gander Antoine Petit Gérard Berry Barbara Demo Jan Vahrenhold Andrew McGettrick Roger Boyle Michèle Drechsler Avi Mendelson Chris Stephenson Carlo Ghezzi Bertrand Meyer. Informatics education : Europe cannot afford to miss the boat. Technical report, Informatics Europe ACM Europe Working Group, Avril 2013.
- [2] Simon Claessens Gaëtan Collart. Rsnap. <http://rsnap.herokuapp.com/>, Mai 2014.
- [3] Brian Harvey Jens Mönig. Snap! reference manual : Version 4.0. Technical report, National Science Foundation, Avril 2014.
- [4] Karen Brennan Michelle Chung Jeff Hawson. *Informatique créative : introduction par le design à la pensée informatique*. MIT, 2011.
- [5] Tech Angels. Gemnasium. <https://gemnasium.com/>, Avril 2014.
- [6] Jeremy Ashkenas. CoffeeScript. <http://coffeescript.org/>, Avril 2014.
- [7] Carolyn Atkinson. Colours for living and learning. http://www.resene.co.nz/homeown/use_colr/coloursforliving.htm, Mai 2014.
- [8] Blockly. blockly : A visual programming editor. <https://code.google.com/p/blockly/>, Avril 2014.
- [9] Blockly. Language design philosophy. <https://code.google.com/p/blockly/wiki/Language>, Avril 2014.
- [10] Jessica Chiang. Shall we learn scratch programming, 2009.
- [11] Code Club. About code club. <https://www.codeclub.org.uk/about>, Avril 2014.
- [12] Code.org. About us. <http://code.org/about>, Avril 2014.
- [13] Code.org. Frequently asked questions. <http://code.org/faq>, Avril 2014.
- [14] Code.org. Teach our k-8 intro to computer science. <http://code.org/educate/20hr>, Avril 2014.
- [15] CoderDojo. About. <http://coderdojo.com/about>, Avril 2014.
- [16] Ruby community. Ruby programming language. <https://www.ruby-lang.org/fr/>, Avril 2014.
- [17] community for Scratch educators. ScratchEd! <http://scratched.media.mit.edu/>, 2014 Avril.
- [18] Rails core team. Getting started with Rails. http://guides.rubyonrails.org/getting_started.html, Avril 2014.

- [19] The Bootstrap core team. Bootstrap. <http://getbootstrap.com/>, Avril 2014.
- [20] cucumber. Gherkin. <https://github.com/cucumber/cucumber/wiki/Gherkin>, Avril 2014.
- [21] Institut de France. L'enseignement de l'informatique en France : Il est urgent de ne plus attendre. Technical report, Académie des sciences, Mai 2013.
- [22] EppO. rolify. <https://github.com/EppO/rolify>, Avril 2014.
- [23] Department for Education. The department for education's statutory guidance publications about schools. <https://www.gov.uk/government/collections/statutory-guidance-schools>, Avril 2014.
- [24] Python Software Foundation. Welcome to Python.org. <https://www.python.org/>, Avril 2014.
- [25] Steve Furber. Shut down or restart ? the way forward for computing in uk schools. Technical report, The royal society, Janvier 2012.
- [26] Travis CI GmbH. Travis CI - free hosted continuous integration platform for the open source community. <https://travis-ci.org/>, Avril 2014.
- [27] Lifelong Kindergarten Group. Scratch - imagine, program, share. <http://scratch.mit.edu/>, Avril 2014.
- [28] David Heinemeier Hansson. Ruby on Rails. <http://rubyonrails.org/>, Avril 2014.
- [29] Aslak Hellesøy. Cucumber - making BDD fun. <http://cukes.info/>, Avril 2014.
- [30] Lemur Heavy Industries. Coveralls - test coverage history & statistics. <https://coveralls.io/>, Avril 2014.
- [31] Jr. Jerry Lee Ford. *Scratch Programming for Teens*. Course Technology PTR, 2009.
- [32] jnicklas. Capybara. <https://github.com/jnicklas/capybara>, Avril 2014.
- [33] Marie Jobin. Scratch à la maternelle. <http://scratched.media.mit.edu/resources/scratch-%C3%A0-la-maternelle>, Avril 2011.
- [34] The jQuery Foundation. Jquery. <http://jquery.com/>, Avril 2014.
- [35] KidsCode. Kidscode. <http://www.kidscode.be/>, Avril 2014.
- [36] nathanl. Authority. <https://github.com/nathanl/authority>, Avril 2014.
- [37] plataformatec. devise. <https://github.com/plataformatec/devise>, Avril 2014.
- [38] Nick Quaranto. RubyGems.org | latest gems. <https://rubygems.org/>, Avril 2014.

- [39] rspec. Rspec. <http://rspec.info/>, Avril 2014.
- [40] Jeremy Scott. *Starting from Scratch An Introduction to Computing Science*. The Royal Society of Edinburgh BCS, 2012.
- [41] Snap! SNAP! (Build Your Own Blocks). <http://snap.berkeley.edu/>, Avril 2014.
- [42] Scratch Team. Languages - translated support materials for scratch 1.4. <http://info.scratch.mit.edu/Languages>, Janvier 2014.
- [43] The Haml Team. Haml. <http://haml.info/>, Avril 2014.
- [44] thoughtbot. factory_girl. https://github.com/thoughtbot/factory_girl, Avril 2014.
- [45] thoughtbot. Paperclip. <https://github.com/thoughtbot/paperclip>, Avril 2014.
- [46] Wikipédia. Baccalauréat scientifique. http://fr.wikipedia.org/wiki/Baccalaur%C3%A9at_scientifique, Avril 2014.
- [47] Wikipédia. Behavior-driven development. https://en.wikipedia.org/wiki/Behaviour-driven_development, Avril 2014.
- [48] Wikipédia. Continuous integration. https://en.wikipedia.org/wiki/Continuous_integration, Avril 2014.
- [49] Wikipédia. Convention over configuration. https://en.wikipedia.org/wiki/Convention_over_Configuration, Avril 2014.
- [50] Wikipédia. Don't repeat yourself. https://en.wikipedia.org/wiki/Don%27t_Repeat_Yourself, Avril 2014.
- [51] Wikipédia. Informatique et sciences du numérique. http://fr.wikipedia.org/wiki/Informatique_et_sciences_du_num%C3%A9rique, Avril 2014.
- [52] Wikipédia. Model-view-controller. <https://en.wikipedia.org/wiki/Model%E2%80%93view%E2%80%93controller>, Avril 2014.
- [53] Wikipédia. Pair programming. https://en.wikipedia.org/wiki/Pair_programming, Avril 2014.
- [54] Wikipédia. Representational state transfer. <https://en.wikipedia.org/wiki/RESTful>, Avril 2014.

ANNEXE A

Référence

A.1 Utilisation de Rsnap

A.1.1 Créer une mission

RSnapMissionsProjets libresProgrammes des étudiants

formulaireSimon Claessens

Missions

N°	Titre	Description			
1	En voiture...	C'est parti pour une course folle. Attention à bien suivre la route. 	Editer	Editer le programme	Supprimer
2	Hélicopter	Seras-tu le plus grand pilote de tout les temps ? Arriveras-tu à boucler le circuit sans trop t'en écarter ? 	Editer	Editer le programme	Supprimer
3	Soyons courtois	Dire "Bonjour" aux passants, c'est quand même la moindre des choses. 	Editer	Editer le programme	Supprimer

Nouvelle mission

RSnap : Gaetan Collart & Simon Claessens

[Qui sommes-nous ?](#)

FIGURE A.1 – Page des missions

RSnap

Missions

Projets libres

Programmes des étudiants

formulaire

Simon Claessens

Créer une mission

Titre

Description

À Texte normal ▾

Gras

Italique

Souligné

La description de la mission avec toutes les informations nécessaire à sa réalisation.

Vidéo youtube

Description courte

À Texte normal ▾

Gras

Italique

Souligné

Description courte d'introduction visible dans la liste des missions

Position

Créer un(e) MissionAnnuler

RSnap : Gaëtan Collart & Simon Claessens : [Qui sommes-nous ?](#)

FIGURE A.2 – Création des informations de la mission

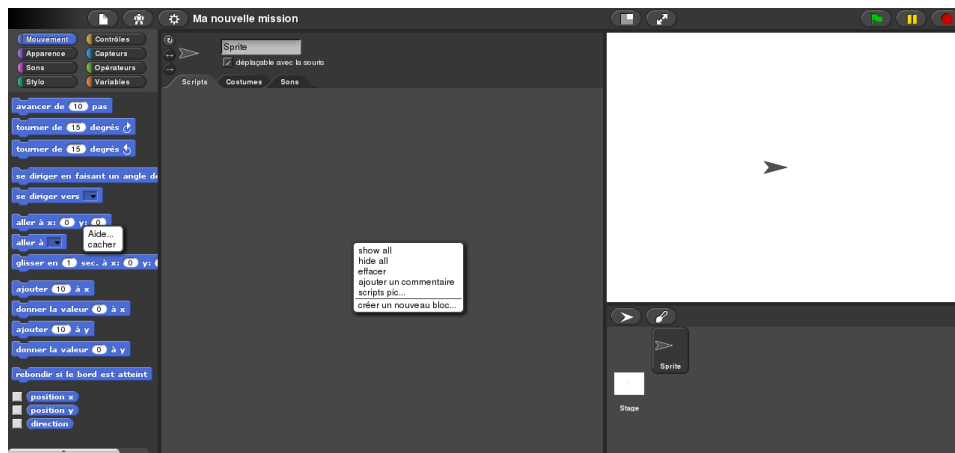


FIGURE A.3 – Création du programme de la mission

A.1.2 Ordonner les missions

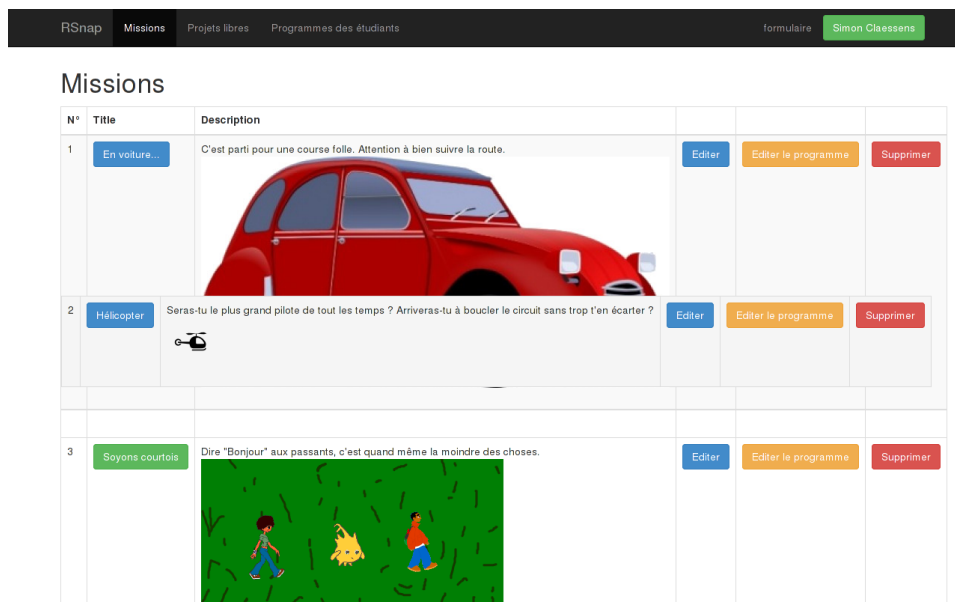


FIGURE A.4 – Ordonner les missions

A.1.3 Corriger une mission

RSnap Missions Projets libres Programmes des étudiants formulaire Simon Claessens		
Programmes		
Mission	Utilisateur	
En voiture...	Edouard Michiels	Supprimer
Soyons courtois	Edouard Michiels	Supprimer
En voiture...	Thibault Van Raemdonck	Supprimer
Soyons courtois	Gaetan Collart	Supprimer
Helicopter	ga co	Supprimer
En voiture...	ga co	Supprimer
En voiture...	infuse infuse	Supprimer
En voiture...	NathaSam dupon	Supprimer
En voiture...	Aude Wantiez	Supprimer

FIGURE A.5 – Page des programmes des étudiants

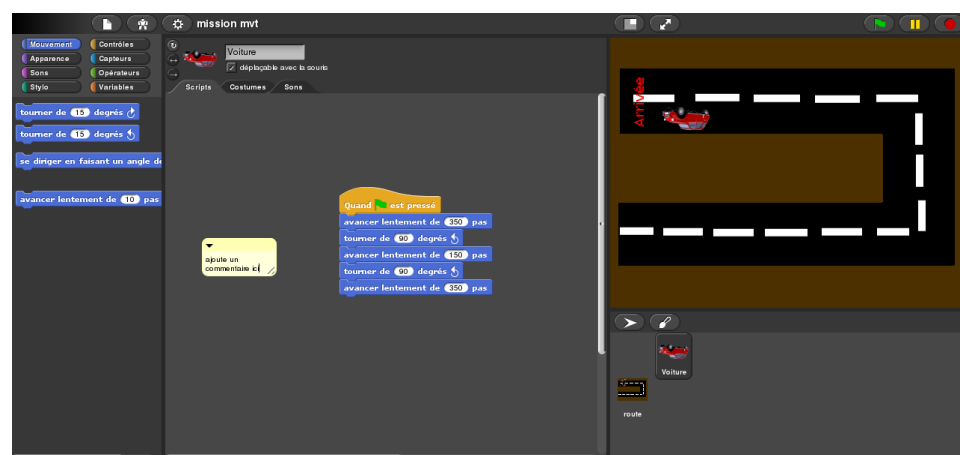


FIGURE A.6 – Correction du programme

A.1.4 Réaliser une mission

RSnap

Missions

Projets libres

formulaire

infuse infuse

Missions

N°	Titre	Description
1	En voiture...	C'est parti pour une course folle. Attention à bien suivre la route. 
2	Hélicopter	Seras-tu le plus grand pilote de tout les temps ? Arriveras-tu à boucler le circuit sans trop t'en écarter ? 
3	Soyons courtois	Dire "Bonjour" aux passants, c'est quand même la moindre des choses. 
4	Tu ne m'attraperas jamais...	C'est la guerre dans le jardin. Le chien et le chat se cherchent l'un, l'autre. Qui sera le vainqueur ? 

RSnap : Gaetan Collart & Simon Claessens - [Qui sommes-nous ?](#)

FIGURE A.7 – Page des missions

RSnap Missions Projets libres formulaire **infuse infuse**

Mission : Soyons courtois

Dans cette mission tu va croiser Julie et Mario. Il te dirons bonjour à chaque fois, fais de même!

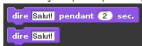


Tu as donc deux choses à savoir :

- comment dire "bonjour"
- comment savoir quand tu es près de Julie ou de Mario

Dire "bonjour"

Commençons par le plus simple. Tu trouveras dans le menu "Apparence" différent bloc pour parler.



Touché !

Pour savoir si Julie ou Mario te touche, il faut utiliser le bloc  pour connaître si on touche un autre joueur. Ce bloc s'utilise de la même manière que ce que tu as déjà du faire pour l'hélicoptère.



A toi de jouer !

Réaliser cette mission

Retour

RSnap : Gaetan Collart & Simon Claessens : [Oui sommes-nous ?](#)

FIGURE A.8 – Description de la mission

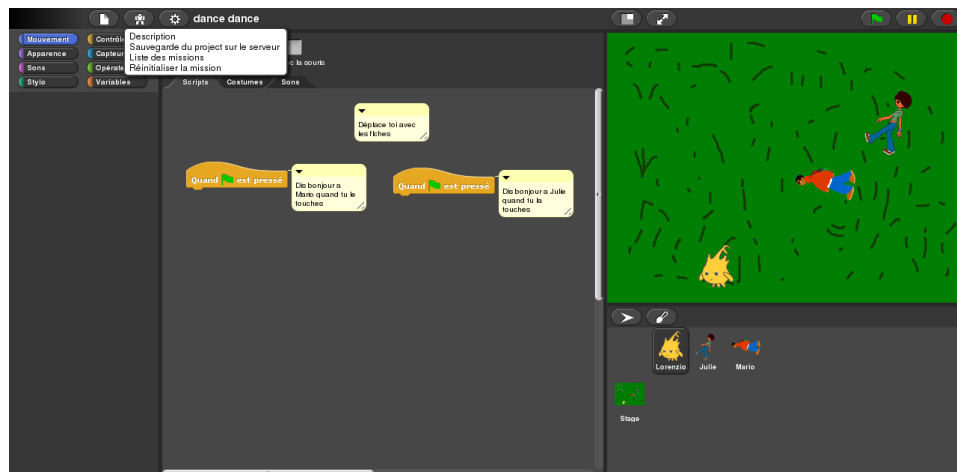


FIGURE A.9 – Réalisation de la mission

A.1.5 Réaliser un projet libre

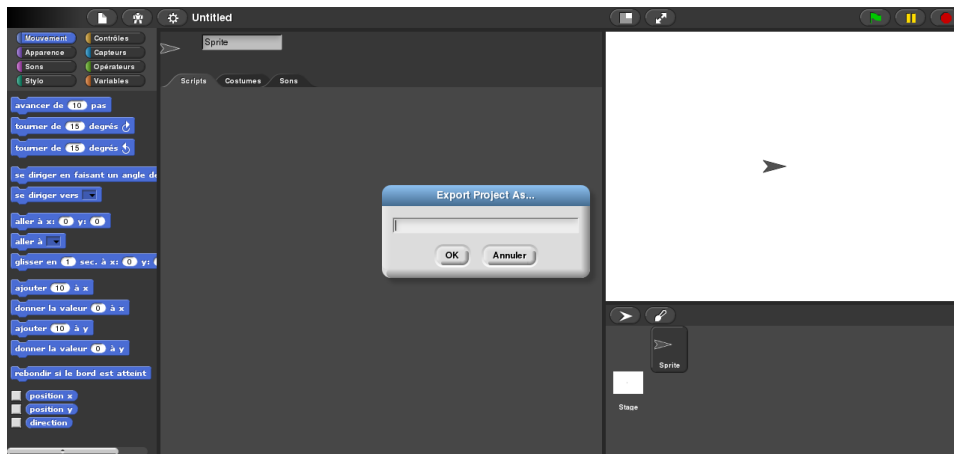


FIGURE A.10 – Page des projets libres

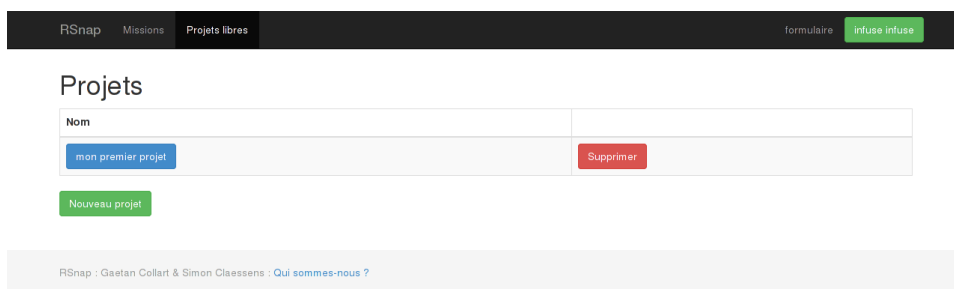


FIGURE A.11 – Création du programme du projet libre

A.2 Réponses au formulaire des enfants

A.3 Réponses au formulaire des professeurs

TABLE A.1 – Réponses aux questions d'ordre général

Je suis à ...	Je suis en ...	Je savais utiliser un ordinateur avant de venir ?	Je sais mieux utiliser un ordinateur maintenant ?	Je savais ce qu'était la programmation avant de venir ?	Je comprends mieux ce qu'est la programmation maintenant ?	J'ai encore des choses à vous dire.
Collège Cardinal Mercier	le secondaire	3	3	4	3	nan
Collège Saint Joseph	le secondaire	3	4	2	4	
Collège Saint-Joseph Chimay	le secondaire	3	1	3	3	
college st joseph chimay	le secondaire	2	2	1	3	
au college saint joseph de chimay	le secondaire	4	4	1	4	
college saint joseph	le secondaire	2	3	1	3	bon courage pour votre fin d'année !!! félicitation
collège saint joseph à chimay	le secondaire	3	3	1	4	
collège st Joseph chimay	le secondaire	2	3	2	3	
Saint Joseph Chimay	le secondaire	3	3	1	4	c etais cool
Collège St Joseph	le secondaire	2	3	1	3	

TABLE A.2 – Réponses aux questions d'ordre général (suite)

Je suis à ...	Je suis en ...	Je savais utiliser un ordinateur avant de venir ?	Je sais mieux utiliser un ordinateur maintenant ?	Je savais ce qu'était la programmation avant de venir ?	Je comprends mieux ce qu'est la programmation maintenant ?	J'ai encore des choses à vous dire.
Collège St Joseph	1e secondaire	4	4	0	0	amusant, des missions plus faciles que les autres, beaucoup de réflexions.
collège saint joseph	1e secondaire	4	1	1	3	
college saint joseph	1e secondaire	4	1	2	4	
st-joseph	1e secondaire	4	4	1	4	c'était trop cool on ces bien amuser merci!!!!!!!!!!!!!!
Collège Saint Joseph	1e secondaire	4	1	4	1	non
college saint joseph 1c8	1e secondaire	4	3	2	4	que c'était en partit quand même bien malgré le soucis
Chimay	1e secondaire	3	2	1	2	

TABLE A.3 – Réponses aux questions d'ordre général (suite 2)

Je suis à ...	Je suis en ...	Je savais utiliser un ordinateur avant de venir ?	Je sais mieux utiliser un ordinateur maintenant ?	Je savais ce qu'était la programmation avant de venir ?	Je comprends mieux ce qu'est la programmation maintenant ?	J'ai encore des choses à vous dire.
collège saint joseph à chimay	1e secondaire	2	2	1	4	un peu plus de mission dans ce genre et des mission un peu plus compliquer pour les plus grand et des plus facile pour les plus petit
collège st Joseph chimay	1e secondaire	4	1	2	3	
au college saint josephe de chimay	1e secondaire	2	1	1	1	
st marie	6e primaire	4	4	0	4	
ecole st marie bousval	6e primaire	3	3	1	4	
sainte marie rendeux	6e primaire 6e primaire	4 2	1 2	3 2	3 3	

TABLE A.4 – Réponses aux questions d'ordre général (suite 3)

Je suis à ...	Je suis en ...	Je savais utiliser un ordinateur avant de venir ?	Je sais mieux utiliser un ordinateur maintenant ?	Je savais ce qu'était la programmation avant de venir ?	Je comprends ce qu'est la programmation maintenant ?	J'ai encore des choses à vous dire.
ecole Ste Marie	6e primaire	4	1	1	4	PLUS VITE CET BIIIIIIIIIIP D'ORDI- NATEUR!!!
Sainte-Marie	6e primaire	4	2	3	4	non juste les bug
école St-Marie	6e primaire	4	4	1	3	s etais trop COOL et on
Ecole Sainte Marie	6e primaire	2	4	1	3	sais bien amusé
Collège du Biéreau	6e primaire	4	1	4	4	
aro	6e primaire	3	2	1	3	
aro	6e primaire	4	4	1	4	
aro	6e primaire	4	1	0	4	
aro paul delvaux ottignies	6e primaire	2	3	1	1	c est trop dur mais sava
A.R.O.	6e primaire	4	1	3	4	
ARO	6e primaire	4	2	3	3	
athénée royal paul delvaux ottignies	6e primaire	4	1	2	4	

TABLE A.5 – Réponses relatives à la mission "En voiture"

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
4	4	3	4	J'ai des remarques à transmettre. C'était fort bien sympathique :) On aimerait recommencer un jour ! j ai pas tres bien compris les donnees sur le cote de l ecran rien est a ameliorer c'était génial!!! une meilleur explication aurait été plus énable en tout cas... J ai bien aimé la mission de la voiture j ai tous aime :),je n ai rien détesté,tous,rien c etais cool j ai bien aimé utiliser les bloques et faire avancer la voiture et surtout quand on a réussit on etait content bref on a bien aimer merci :)
3	2	4	4	
4	3	4	4	
2	1	1	4	
4	4	3	4	
4	4	4	4	
2	2	2	3	
3	3	4	4	
4	4	3	4	
4	3	4	3	
4	3	3	3	
4	4	2	3	
4	4	3	4	
4	4	3	4	

TABLE A.6 – Réponses relatives à la mission "En voiture" (suite)

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
4	3	3	3	au depart on comprenait pas trop mais maintenant ces beaucoup plus facile j ai tout aimer ;) ;) C'était bien on s'est bien amusée mais il y a des missions qu'on n'a pas trop aimé car on ne savait pas ce qu'il fallait faire LA LOGIQUE ÉTAIS BIEN MAI SELLA ÉTAIT TRÈS TRÈS DURE NOUS AVONS PAS SU RÉPONDRE SANS LAIDE DES ACOMPAGNATEUR nous avons trouver sa marrant , au début c'était dur mais a la fin sa allait c est bien expliquer je n ai rien a dire j'ai adorée la mission etait bien amusante et chouette a realiser les lags etaient genant
4	4	4	4	
4	4	4	3	
3	3	3	4	
4	2	1	3	
2	3	3	3	
3	2	1	2	
4	4	4	4	
4	3	3	3	
3	4	4	3	

TABLE A.7 – Réponses relatives à la mission "En voiture" (suite 2)

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	
3	4	4	3	J'ai des remarques à transmettre.
4	4	4	3	
4	4	4	2	
4	3	4	4	la rapidité de l'ordinateur des bug nous avons aimé parce que se n'étais pas trop difficile et que nous nous sommes bien amusée votre ordinateur est tres tres lent et animation etais cool
3	1	2	3	
4	4	4	4	
4	3	3	3	c estait bien sauf quand sa fesait boum Très chouette on a bien rigolé Ce projet est une très bonne idée mais les explications n'étaient pas claires et programme buggait on a eu quelque difficulté a la premiere mission mais apres on
4	4	4	4	
2	1	3	3	
4	3	3	4	
3	2	1	3	
3	3	1	1	

TABLE A.8 – Réponses relatives à la mission "Hélicoptère"

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
3	3	4	3	Rien à dire :D apres qu on nous ai expliquer nous avons compris ca fait bouun quand on touche la ligne vert c'était bien mais je suis rester bloquer car je n'avais pas la bonne couleur :) on ne nous a pas bien expliquer se qu il fallait faire... a part ca rien n'a dire de grave J AI AIME c etais cool un peu compliquer a comprendre javate
4	3	4	3	
3	2	4	3	
2	3	2	1	
4	3	2	4	
3	3	3	3	
2	2	2	2	
3	4	4	3	
4	4	4	4	
3	4	3	4	
4	4	1	4	
3	2	2	2	
4	3	2	3	
4	4	4	4	
3	3	4	3	

TABLE A.9 – Réponses relatives à la mission "Hélicoptère" (suite)

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
2	2	2	2	C'était compliqué on a du appeler quelqu'un PLUTÔT COOL QU'AND ON A COMPRIS MAI LES EXPLICATION NE TAIT pas FACILE A COMPRENDRE LA PREMIERES FOI
4	3	3	3	c'était sympas facile bien expliquer
2	4	3	4	aucune remarque a dire parfait
1	2	1	2	on la fait que 1 fois et on a eu bon
4	4	4	4	il faut juste ameliorer la vitesse de l'helicoptere
4	4	4	3	
4	3	4	4	
1	1	1	1	faites que se sois plus FACILE
4	4	4	3	aussi bug
4	3	4	3	nous avons aimé car il y avait un peu de difficulté et on a bien rigoler
2	1	1	3	
4	4	4	4	

TABLE A.10 – Réponses relatives à la mission "Hélicoptère" (suite 2)

J'ai aimé réaliser cette mission ?	Avec les expli- cations données et le texte décri- vant la mission, j'ai compris le but de la mis- sion ?	Avec les expli- cations données et le texte dé- crivant la mis- sion, j'ai su par où commencer ?	Avec les expli- cations données et le texte dé- crivant la mis- sion, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
0	0	0	0	
1	1	1	1	
3	3	3	3	
0	0	0	0	
0	0	0	0	
2	1	1	1	

TABLE A.11 – Réponses relatives à la mission "Soyons courtois"

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	
0	0	0	0	On a bien aimé en général !
4	4	4	4	
4	4	3	4	
4	3	2	4	
4	2	3	4	je n'avais pas trop compris mais des personnes m'ont expliqué et j'ai compris !! assez facile !!
4	2	2	3	
4	4	4	4	
4	4	3	4	
3	4	3	4	c etais cool
4	2	2	2	
4	4	4	4	
4	4	2	3	
2	1	2	3	un peut trop facile dur a comprendre
1	3	2	3	
4	4	4	4	
2	2	2	1	
3	3	3	3	setais trop fastoche je n'ai pas compris le but On a eu un beug car ils ont pas mit parfait JE NES RIEN A DIRE
3	3	3	3	
2	1	1	1	
0	0	0	0	

TABLE A.14 – Réponses relatives à la mission "Tu ne m'attraperas pas" (suite)

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	
1	1	0	1	On a pas su la faire car quelqu'un est venu mais on a pas compris SET AI HORRIBLEMENT DUR cette mission est plutôt dur on ne comprend pas très bien mais c etait marrant surtout avec les personnage
1	1	1	1	
3	3	3	3	
1	1	1	1	
4	4	4	4	
3	3	3	3	
0	0	0	0	
0	0	0	0	
0	0	0	0	
0	0	0	0	
0	0	0	0	
0	0	0	0	
0	0	0	0	
0	0	0	0	
4	4	4	4	
0	0	0	0	

TABLE A.15 – Réponses relatives à la mission "Tu ne m'attraperas pas" (suite 2)

J'ai aimé réaliser cette mission ?	Avec les explications données et le texte décrivant la mission, j'ai compris le but de la mission ?	Avec les explications données et le texte décrivant la mission, j'ai su par où commencer ?	Avec les explications données et le texte décrivant la mission, j'ai réalisé la mission ?	J'ai des remarques à transmettre.
3	2	3	3	c etait cool
4	4	4	4	
4	4	4	4	
2	3	3	0	

TABLE A.16 – Réponses données par les professeurs

Qualité de l'animation (1 à 7)	Remarques sur l'animation	Remarques sur les mises	Remarques générales
6	excellente initiative. les enfants ont bien accroché.		excellente activité de logique.....à développer
6	très bien. ProgreSSION adaptée afin de permettre aux "moins motivés" et aux "moins doués" d'entrer dans l'univers de la programmation	attrayantes ; adaptées à des jeunes de 12-13 ans.	super.
7	- Bonne mise en place des consignes.- Bon suivi dans les sous-groupes.- Bonne gestion du groupe.	- Bons choix.- Evolution dans la difficulté / arrivées de nouvelles possibilités de déplacements par étapes.- Possibilité de dépacement.	- Excellent projet de TFE. A développer !- Je pense utiliser ce programme dans mes cours.

